0. Libraries
1. Data
2. Price series plots
3. Log-returns
3b. Risk-free rate — US T-bill 3-month (FRED: DTB3)
4. Descriptive statistics
5. Histograms
6. Normality test (Jarque-Bera test)
7. Unit root tests (ADF test)
8. ACF / PACF
9. ARMA model selection (AIC)
10. ARCH effect test (LM test)
11. Univariate GARCH models
12. Standardised residual diagnostics
13. DCC-GARCH
14. Dynamic correlation & covariance plots
15. GO-GARCH
16. Forecast evaluation — variance & correlation
17. Core VaR / ES / Backtest functions
18. Optimized Portfolio
20. DCC-GARCH — VaR & ES (alpha = 1%, 2.5%, 5%)
21. GO-GARCH — VaR & ES (alpha = 1%, 2.5%, 5%)
22. Historical Simulation — VaR & ES
23. Summary backtest table (by alpha)
24. FZ Score & Violation rate plots
25. VaR/ES Ranking (averaged over all alpha levels)
26. Diebold-Mariano test for FZ loss differences
27. Portfolio performance analysis
28. Backtesting report

Market Risk Management:Multivariate GARCH Approaches

Code

Minh Duc Nguyen

2025-12-22

0. Libraries

Hide

```
pkgs <- c(
  "quantmod", "xts", "zoo", "foreign", "car", "carData",
  "lmtest", "dynlm", "sandwich", "stargazer", "dplyr",
  "systemfit", "caTools", "readxl", "AER", "stringr",
  "mlogit", "nnet", "caret", "reshape2", "survival",
  "forecast", "Metrics", "tseries", "tsutils", "ggplot2",
  "urca", "rugarch", "Information", "FinTS", "TTR",
  "vars", "rmgarch", "MTS", "broom", "quadprog",
  "summarytools", "knitr", "kableExtra", "MCS", "htmltools"
)
invisible(lapply(pkgs, function(p) {
  if (!requireNamespace(p, quietly = TRUE)) install.packages(p)
  library(p, character.only = TRUE)
}))
```

1. Data

Hide

```
# Tickers
ticker_sp500 <- "SPY"    # S&P 500 ETF
ticker_tb    <- "TLT"    # 20+ Year Treasury Bond ETF
ticker_gold  <- "GLD"    # Gold ETF
ticker_oil   <- "USO"    # Oil ETF

#Date range
date_start <- "2021-01-01"
date_end   <- "2025-12-31"
```

Hide

```
# Download closing prices
get_close <- function(ticker, start, end) {
  tryCatch({
    x <- getSymbols(ticker, src = "yahoo",
                    from = start, to = end,
                    auto.assign = FALSE)
    Ad(x)
  }, error = function(e) {
    stop(sprintf("Cannot download %s: %s", ticker, conditionMessage(e)))
  })
}

raw_sp500 <- get_close(ticker_sp500, date_start, date_end)
raw_tb    <- get_close(ticker_tb,    date_start, date_end)
raw_gold  <- get_close(ticker_gold,  date_start, date_end)
raw_oil   <- get_close(ticker_oil,   date_start, date_end)
```

Hide

```
# Merge by date, keep only dates with no missing across all 4 variables
prices <- merge(raw_sp500, raw_tb, raw_gold, raw_oil,
                join = "inner")
colnames(prices) <- c("sp500", "tb", "gold", "oil")

prices <- na.omit(prices)

cat(sprintf("Date range : %s to %s\n",
            format(index(prices)[1]),
            format(index(prices)[nrow(prices)])))
```

```
## Date range : 2021-01-04 to 2025-12-30
```

Hide

```
cat(sprintf("Observations: %d days\n", nrow(prices)))
```

```
## Observations: 1254 days
```

Hide

```
cat(sprintf("Missing values: %d\n", sum(is.na(prices))))
```

```
## Missing values: 0
```

Hide

```
print(head(prices))
```

```
##               sp500       tb   gold   oil
## 2021-01-04 343.3191 133.7556 182.33 32.38
## 2021-01-05 345.6837 132.7623 182.87 33.94
## 2021-01-06 347.7503 130.0369 179.90 34.11
## 2021-01-07 352.9171 128.8907 179.48 34.53
## 2021-01-08 354.9279 128.4747 173.34 35.43
## 2021-01-11 352.5354 128.2625 173.00 35.26
```

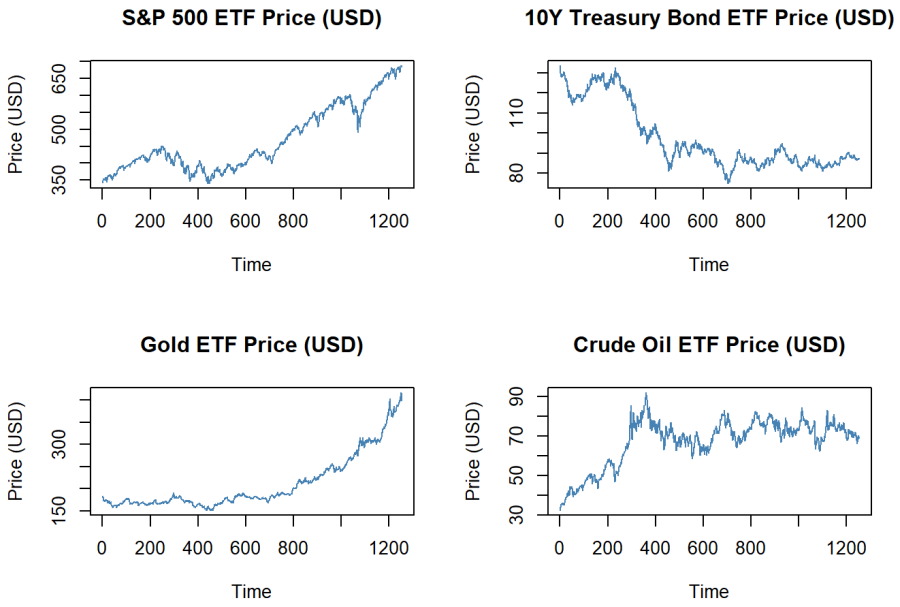
Hide

```
# Extract vectors
sp500 <- as.numeric(prices$sp500)
tb    <- as.numeric(prices$tb)
gold  <- as.numeric(prices$gold)
oil   <- as.numeric(prices$oil)
```

2. Price series plots

Hide

```
par(mfrow = c(2, 2))
plot.ts(sp500, main = "S&P 500 ETF Price (USD)",              col = "steelblue", ylab = "Price (USD)")
plot.ts(tb,    main = "10Y Treasury Bond ETF Price (USD)",    col = "steelblue", ylab = "Price (USD)")
plot.ts(gold,  main = "Gold ETF Price (USD)",                 col = "steelblue", ylab = "Price (USD)")
plot.ts(oil,   main = "Crude Oil ETF Price (USD)",            col = "steelblue", ylab = "Price (USD)")
```



Hide

```
par(mfrow = c(1, 1))
```

3. Log-returns

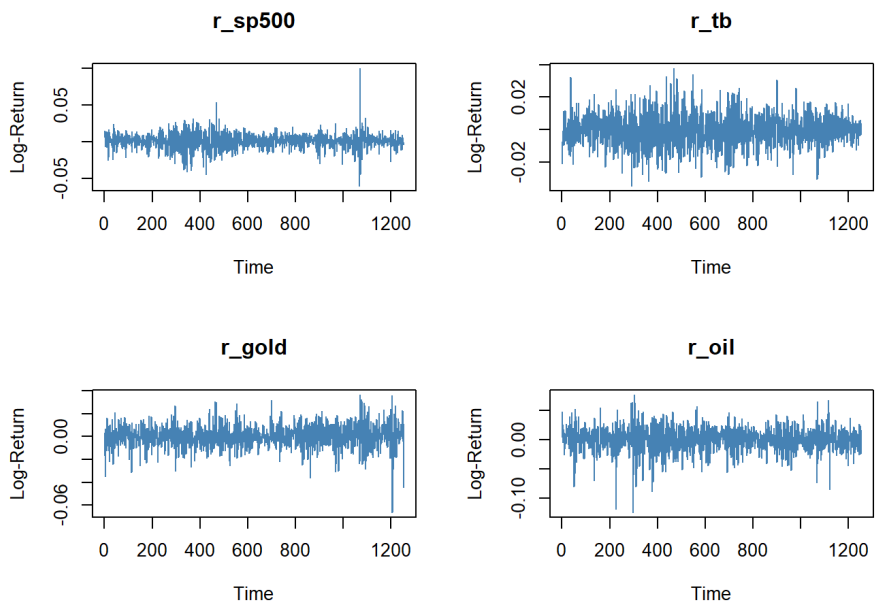
Hide

```
r_gold  <- diff(log(gold))
r_oil   <- diff(log(oil))
r_tb    <- diff(log(tb))
r_sp500 <- diff(log(sp500))

returns <- cbind(r_sp500, r_tb, r_gold, r_oil)
head(returns)
```

```
##            r_sp500          r_tb       r_gold        r_oil
## [1,]  0.0068637522 -0.0074541824  0.002957249  0.047053238
## [2,]  0.0059606435 -0.0207421966 -0.016374384  0.004996395
## [3,]  0.0147483430 -0.0088532206 -0.002337350  0.012237862
## [4,]  0.0056814085 -0.0032330010 -0.034808795  0.025730451
## [5,] -0.0067635903 -0.0016530840 -0.001963368 -0.004809798
## [6,]  0.0002111987 -0.0007951235  0.006453094  0.017988209
```

Hide

```
par(mfrow = c(2, 2))
plot.ts(r_sp500, main = "r_sp500", ylab = "Log-Return", col = "steelblue")
plot.ts(r_tb,    main = "r_tb",    ylab = "Log-Return", col = "steelblue")
plot.ts(r_gold,  main = "r_gold",  ylab = "Log-Return", col = "steelblue")
plot.ts(r_oil,   main = "r_oil",   ylab = "Log-Return", col = "steelblue")
```



Hide

```
par(mfrow = c(1, 1))
```

3b. Risk-free rate — US T-bill 3-month (FRED: DTB3)

Hide

```
# Download DTB3 from FRED.
rf_daily_series <- tryCatch({
  getSymbols("DTB3", src = "FRED", auto.assign = FALSE)
}, error = function(e) {
  warning("Could not download DTB3 from FRED. Falling back to rf = 0. ",
          conditionMessage(e))
  NULL
})

if (!is.null(rf_daily_series)) {
  # DTB3 in % per annum -> convert to daily decimal
  rf_daily_series <- rf_daily_series / 100 / 252

  # Build date index aligned to returns rows
  if ("Date" %in% colnames(data)) {
    ret_dates <- as.Date(data$Date[-1])          # diff() removes first row
  } else {
    # Synthetic trading-day calendar starting 2000-01-03
    all_days <- seq(as.Date("2000-01-03"), by = "day",
                    length.out = nrow(returns) * 3)
    trading  <- all_days[!weekdays(all_days) %in% c("Saturday", "Sunday")]
    ret_dates <- trading[1:nrow(returns)]
  }

  # For each return date, use the most recent T-bill rate on or before that date
  # (strictly no-lookahead: rate at t-1 is used when deciding weights at t-1)
  rf_vec <- sapply(ret_dates, function(d) {
    idx <- which(index(rf_daily_series) <= d)
    if (length(idx) == 0L) return(0)
    as.numeric(rf_daily_series[max(idx)])
  })
  rf_vec[is.na(rf_vec)] <- 0

  cat(sprintf("DTB3 loaded:  %d obs | %s to %s
",
              nrow(rf_daily_series),
              format(min(index(rf_daily_series))),
              format(max(index(rf_daily_series)))))
  cat(sprintf("rf_vec:       %d obs aligned to returns
", length(rf_vec)))
  cat(sprintf("rf daily:     min=%.8f  max=%.8f  mean=%.8f
",
              min(rf_vec), max(rf_vec), mean(rf_vec)))
  cat(sprintf("rf annualised: %.4f%%
", mean(rf_vec) * 252 * 100))

} else {
  rf_vec <- rep(0, nrow(returns))
  cat("rf = 0 (fallback — DTB3 unavailable).
")
}
```

```
## DTB3 loaded:  18845 obs | 1954-01-04 to 2026-03-27
## rf_vec:       1253 obs aligned to returns
## rf daily:     min=0.00000000  max=0.00024762  mean=0.00012588
## rf annualised: 3.1721%
```

Hide

```
# Scalar summaries (no lookahead: each uses only data from the relevant window)
rf_is_mean  <- mean(rf_vec[1:1461], na.rm = TRUE)
rf_oos_mean <- mean(rf_vec[(nrow(returns) - 249):nrow(returns)], na.rm = TRUE)
rf_daily    <- rf_is_mean    # scalar alias for in-sample use

cat(sprintf("
rf_is_mean  = %.8f / day  (%.4f%% ann)
",
            rf_is_mean,  rf_is_mean  * 252 * 100))
```

```
##
## rf_is_mean  = 0.00012588 / day  (3.1721% ann)
```

Hide

```
cat(sprintf("rf_oos_mean = %.8f / day  (%.4f%% ann)
",
            rf_oos_mean, rf_oos_mean * 252 * 100))
```

```
## rf_oos_mean = 0.00004567 / day  (1.1508% ann)
```

4. Descriptive statistics

Hide

```
data_list <- list(r_sp500 = r_sp500, r_tb = r_tb, r_gold = r_gold, r_oil = r_oil)

descr(returns, round.digits = 6)
```

```
## Descriptive Statistics
##
##                          r_gold         r_oil       r_sp500          r_tb
## ----------------- ------------- ------------- ------------- -------------
##              Mean      0.000625      0.000612      0.000551     -0.000341
##           Std.Dev      0.009849      0.021016      0.010764      0.010124
##               Min     -0.066427     -0.123962     -0.060327     -0.034780
##                Q1     -0.004667     -0.011689     -0.004526     -0.007077
##            Median      0.000618      0.001471      0.000760     -0.000223
##                Q3      0.006171      0.013653      0.006288      0.005934
##               Max      0.036323      0.076115      0.099863      0.037752
##               MAD      0.008023      0.018761      0.008069      0.009624
##               IQR      0.010839      0.025343      0.010813      0.013011
##                CV     15.763133     34.321493     19.519191    -29.679932
##          Skewness     -0.362899     -0.611474      0.124250      0.026885
##       SE.Skewness      0.069116      0.069116      0.069116      0.069116
##          Kurtosis      2.695813      2.637585      8.039738      0.420799
##           N.Valid   1253.000000   1253.000000   1253.000000   1253.000000
##                 N   1253.000000   1253.000000   1253.000000   1253.000000
##         Pct.Valid    100.000000    100.000000    100.000000    100.000000
```

Hide

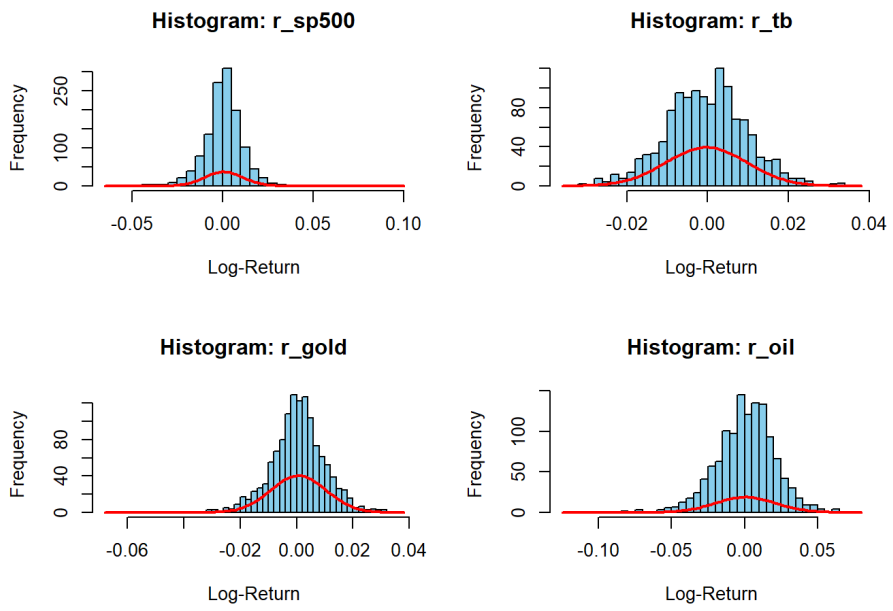
```
invisible(lapply(names(data_list), function(nm) {
  r <- data_list[[nm]]
  cat(sprintf("\n--- %s ---  Pos: %d | Neg: %d | Zero: %d\n",
              nm, sum(r > 0), sum(r < 0), sum(r == 0)))
}))
```

```
##
## --- r_sp500 ---  Pos: 686 | Neg: 565 | Zero: 2
##
## --- r_tb ---  Pos: 615 | Neg: 635 | Zero: 3
##
## --- r_gold ---  Pos: 673 | Neg: 579 | Zero: 1
##
## --- r_oil ---  Pos: 667 | Neg: 582 | Zero: 4
```

5. Histograms

Hide

```
par(mfrow = c(2, 2))
for (nm in names(data_list)) {
  r <- data_list[[nm]]
  hist(r, main = paste("Histogram:", nm), col = "skyblue",
       xlab = "Log-Return", prob = FALSE, breaks = 50)
  curve(dnorm(x, mean(r, na.rm = TRUE), sd(r, na.rm = TRUE)),
        add = TRUE, col = "red", lwd = 2)
}
```



Hide

```
par(mfrow = c(1, 1))
```

6. Normality test (Jarque-Bera test)

Hide

```
invisible(lapply(names(data_list), function(nm) {
  cat(sprintf("\n=== Jarque-Bera: %s ===\n", nm))
  print(jarque.bera.test(data_list[[nm]]))
}))
```

```
##
## === Jarque-Bera: r_sp500 ===
##
##  Jarque Bera Test
##
## data:  data_list[[nm]]
## X-squared = 3392.7, df = 2, p-value < 2.2e-16
##
##
## === Jarque-Bera: r_tb ===
##
##  Jarque Bera Test
##
## data:  data_list[[nm]]
## X-squared = 9.6377, df = 2, p-value = 0.008076
##
##
## === Jarque-Bera: r_gold ===
##
##  Jarque Bera Test
##
## data:  data_list[[nm]]
## X-squared = 409.55, df = 2, p-value < 2.2e-16
##
##
## === Jarque-Bera: r_oil ===
##
##  Jarque Bera Test
##
## data:  data_list[[nm]]
## X-squared = 443.96, df = 2, p-value < 2.2e-16
```

7. Unit root tests (ADF test)

ADF test needed for time-series data to check stationarity. Non-stationary data can lead to spurious results in time-series analysis, so it's crucial to confirm that the log-return series are stationary before proceeding with modeling. The ADF test checks for the presence of a unit root, which indicates non-stationarity. We test with different specifications (trend, drift, none) to ensure robustness of the stationarity conclusion across various model assumptions.

Hide

```
invisible(lapply(names(data_list), function(nm) {
  cat(sprintf("\n========== ADF: %s ==========\n", nm))
  for (type in c("trend", "drift", "none")) {
    cat(sprintf("-- type = %s --\n", type))
    print(summary(ur.df(data_list[[nm]], type = type, lags = 0)))
  }
}))
```

```
##
## ========== ADF: r_sp500 ==========
## -- type = trend --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression trend
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.062523 -0.004997  0.000255  0.005755  0.098652
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.801e-04  6.092e-04   0.460    0.646
## z.lag.1     -1.028e+00  2.828e-02 -36.365   <2e-16 ***
## tt           4.502e-07  8.422e-07   0.535    0.593
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01077 on 1249 degrees of freedom
## Multiple R-squared:  0.5143, Adjusted R-squared:  0.5135
## F-statistic: 661.2 on 2 and 1249 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -36.3651 440.8061 661.2089
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau3 -3.96 -3.41 -3.12
## phi2  6.09  4.68  4.03
## phi3  8.27  6.25  5.34
##
## -- type = drift --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression drift
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.062314 -0.005039  0.000301  0.005826  0.098856
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.0005620  0.0003047   1.845   0.0653 .
## z.lag.1     -1.0282026  0.0282694 -36.372   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01077 on 1250 degrees of freedom
## Multiple R-squared:  0.5142, Adjusted R-squared:  0.5138
## F-statistic:  1323 on 1 and 1250 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -36.3715 661.4442
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau2 -3.43 -2.86 -2.57
## phi1  6.43  4.59  3.78
##
## -- type = none --
##
```

```
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression none
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 - 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.061617 -0.004510  0.000854  0.006387  0.099460
##
## Coefficients:
##         Estimate Std. Error t value Pr(>|t|)
## z.lag.1 -1.02553    0.02826  -36.29   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01078 on 1251 degrees of freedom
## Multiple R-squared:  0.5128, Adjusted R-squared:  0.5125
## F-statistic:  1317 on 1 and 1251 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -36.2899
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau1 -2.58 -1.95 -1.62
##
##
## ========== ADF: r_tb ==========
## -- type = trend --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression trend
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.033607 -0.006597  0.000062  0.006378  0.038395
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -9.144e-04  5.729e-04  -1.596    0.111
## z.lag.1     -1.045e+00  2.826e-02 -36.986   <2e-16 ***
## tt           8.996e-07  7.917e-07   1.136    0.256
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01012 on 1249 degrees of freedom
## Multiple R-squared:  0.5227, Adjusted R-squared:  0.522
## F-statistic:   684 on 2 and 1249 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -36.9861 455.9917 683.9875
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau3 -3.96 -3.41 -3.12
## phi2  6.09  4.68  4.03
## phi3  8.27  6.25  5.34
##
## -- type = drift --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression drift
```

```
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.033921 -0.006650  0.000137  0.006384  0.038248
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -0.0003505  0.0002862  -1.225    0.221
## z.lag.1     -1.0442949  0.0282514 -36.964   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01012 on 1250 degrees of freedom
## Multiple R-squared:  0.5222, Adjusted R-squared:  0.5219
## F-statistic:  1366 on 1 and 1250 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -36.9644 683.1827
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau2 -3.43 -2.86 -2.57
## phi1  6.43  4.59  3.78
##
## -- type = none --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression none
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 - 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.034285 -0.007007 -0.000216  0.006036  0.037894
##
## Coefficients:
##         Estimate Std. Error t value Pr(>|t|)
## z.lag.1 -1.04314    0.02824  -36.94   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.01012 on 1251 degrees of freedom
## Multiple R-squared:  0.5217, Adjusted R-squared:  0.5213
## F-statistic:  1364 on 1 and 1251 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -36.9367
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau1 -2.58 -1.95 -1.62
##
##
## ========== ADF: r_gold ==========
## -- type = trend --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression trend
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
```

```
## -0.067541 -0.005392  0.000207  0.005572  0.034829
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept) -7.454e-04  5.562e-04  -1.340  0.18042
## z.lag.1     -1.022e+00  2.829e-02 -36.140  < 2e-16 ***
## tt           2.206e-06  7.708e-07   2.862  0.00428 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.009827 on 1249 degrees of freedom
## Multiple R-squared:  0.5112, Adjusted R-squared:  0.5104
## F-statistic:   653 on 2 and 1249 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -36.1398 435.3627 653.044
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau3 -3.96 -3.41 -3.12
## phi2  6.09  4.68  4.03
## phi3  8.27  6.25  5.34
##
## -- type = drift --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression drift
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.066491 -0.005300 -0.000032  0.005552  0.035777
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.0006329  0.0002791   2.268   0.0235 *
## z.lag.1     -1.0159079  0.0282801 -35.923   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.009855 on 1250 degrees of freedom
## Multiple R-squared:  0.508,  Adjusted R-squared:  0.5076
## F-statistic:  1290 on 1 and 1250 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -35.9231 645.2352
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau2 -3.43 -2.86 -2.57
## phi1  6.43  4.59  3.78
##
## -- type = none --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression none
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 - 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.066003 -0.004633  0.000595  0.006185  0.036388
##
## Coefficients:
##         Estimate Std. Error t value Pr(>|t|)
## z.lag.1 -1.01185    0.02827  -35.79   <2e-16 ***
```

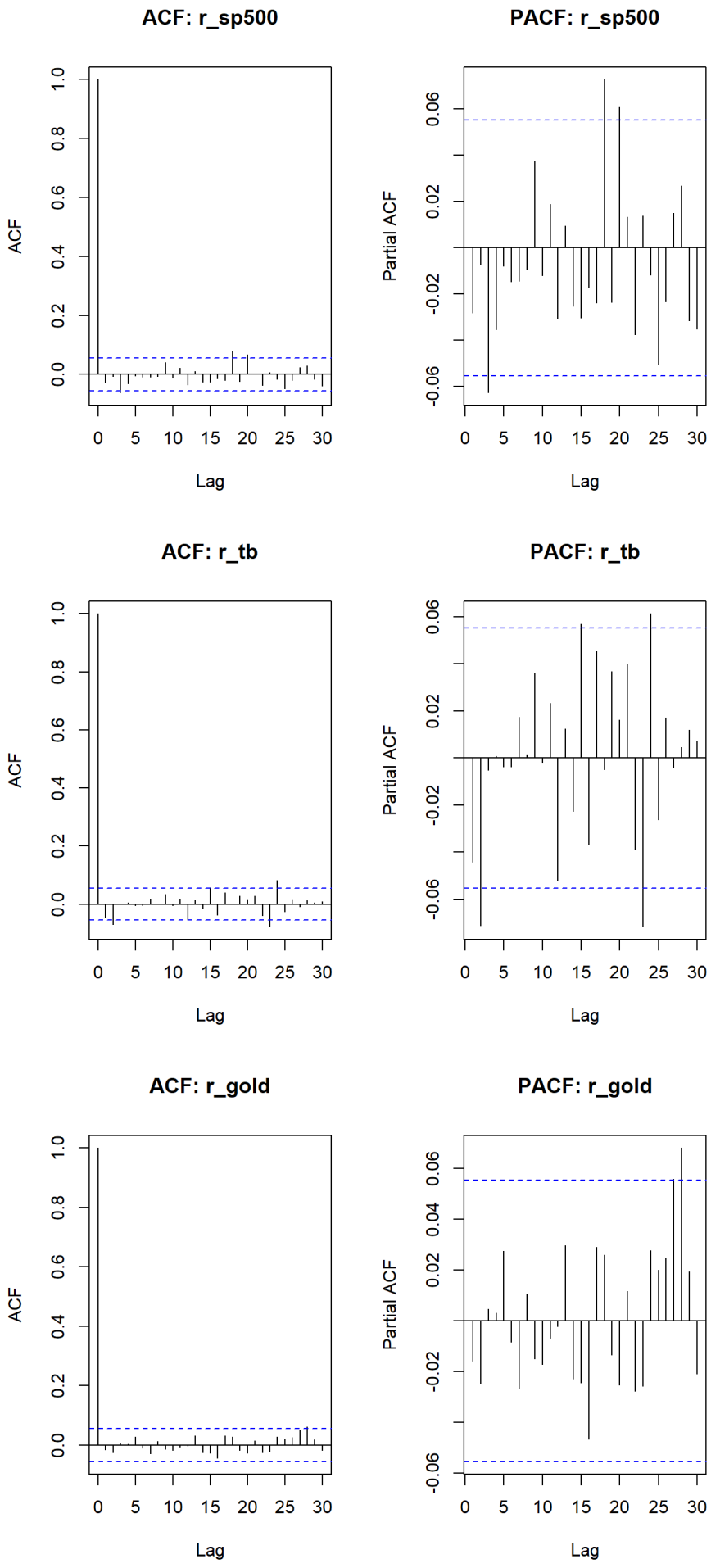
```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.009871 on 1251 degrees of freedom
## Multiple R-squared:  0.5059, Adjusted R-squared:  0.5055
## F-statistic:  1281 on 1 and 1251 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -35.7923
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau1 -2.58 -1.95 -1.62
##
##
## ========== ADF: r_oil ==========
## -- type = trend --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression trend
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1 + tt)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.125971 -0.012050  0.000756  0.013089  0.074897
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.011e-03  1.188e-03   1.693   0.0908 .
## z.lag.1     -9.813e-01  2.824e-02 -34.750   <2e-16 ***
## tt          -2.310e-06  1.642e-06  -1.407   0.1597
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.02098 on 1249 degrees of freedom
## Multiple R-squared:  0.4916, Adjusted R-squared:  0.4908
## F-statistic: 603.8 on 2 and 1249 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -34.7497 402.52 603.7781
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau3 -3.96 -3.41 -3.12
## phi2  6.09  4.68  4.03
## phi3  8.27  6.25  5.34
##
## -- type = drift --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression drift
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 + 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.125270 -0.012333  0.000909  0.013087  0.075656
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  0.0005627  0.0005934   0.948    0.343
## z.lag.1     -0.9795638  0.0282231 -34.708   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.02099 on 1250 degrees of freedom
```

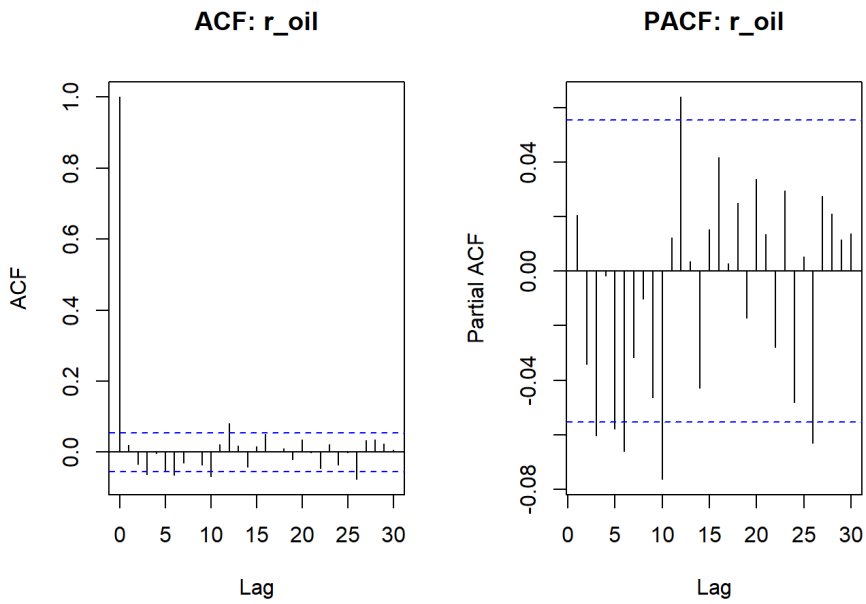
```
## Multiple R-squared:  0.4908, Adjusted R-squared:  0.4904
## F-statistic:  1205 on 1 and 1250 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -34.7078 602.3186
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau2 -3.43 -2.86 -2.57
## phi1  6.43  4.59  3.78
##
## -- type = none --
##
## ###############################################
## # Augmented Dickey-Fuller Test Unit Root Test #
## ###############################################
##
## Test regression none
##
##
## Call:
## lm(formula = z.diff ~ z.lag.1 - 1)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.124736 -0.011766  0.001469  0.013671  0.076223
##
## Coefficients:
##         Estimate Std. Error t value Pr(>|t|)
## z.lag.1 -0.97879    0.02821   -34.7   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.02099 on 1251 degrees of freedom
## Multiple R-squared:  0.4904, Adjusted R-squared:   0.49
## F-statistic:  1204 on 1 and 1251 DF,  p-value: < 2.2e-16
##
##
## Value of test-statistic is: -34.6963
##
## Critical values for test statistics:
##       1pct  5pct 10pct
## tau1 -2.58 -1.95 -1.62
```

8. ACF / PACF

Hide

```
invisible(lapply(names(data_list), function(nm) {
  par(mfrow = c(1, 2))
  acf(data_list[[nm]],  main = paste("ACF:", nm))
  pacf(data_list[[nm]], main = paste("PACF:", nm))
  par(mfrow = c(1, 1))
}))
```


ACF: r_sp500
PACF: r_sp500
ACF: r_tb
PACF: r_tb
ACF: r_gold
PACF: r_gold
ACF
Partial ACF
Lag



9. ARMA model selection (AIC)

Hide

```
arma_sp500 <- auto.arima(r_sp500, ic = "aic")
arma_tb    <- auto.arima(r_tb,    ic = "aic")
arma_gold  <- auto.arima(r_gold,  ic = "aic")
arma_oil   <- auto.arima(r_oil,   ic = "aic")
invisible(lapply(list(arma_sp500, arma_tb, arma_gold, arma_oil), summary))
```

10. ARCH effect test (LM test)

ARCH test needed for checking the presence of autoregressive conditional heteroskedasticity (ARCH) effects in the residuals of the fitted ARMA models. Detecting ARCH effects is crucial because it indicates that the variance of the errors is not constant over time, which violates the assumptions of standard linear regression models. If ARCH effects are present, it justifies the use of GARCH-type models that can capture time-varying volatility, leading to more accurate modeling and forecasting of financial returns.

Hide

```
res_list <- list(
  r_sp500 = arma_sp500$residuals, r_tb  = arma_tb$residuals,
  r_gold  = arma_gold$residuals,  r_oil = arma_oil$residuals
)
invisible(lapply(names(res_list), function(nm) {
  cat(sprintf("\n=== ARCH Test: %s ===\n", nm))
  print(ArchTest(res_list[[nm]]^2, lags = 10))
}))
```

```
##
## === ARCH Test: r_sp500 ===
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  res_list[[nm]]^2
## Chi-squared = 49.295, df = 10, p-value = 3.597e-07
##
##
## === ARCH Test: r_tb ===
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  res_list[[nm]]^2
## Chi-squared = 10.368, df = 10, p-value = 0.4088
##
##
## === ARCH Test: r_gold ===
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  res_list[[nm]]^2
## Chi-squared = 0.51295, df = 10, p-value = 1
##
##
## === ARCH Test: r_oil ===
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  res_list[[nm]]^2
## Chi-squared = 22.2, df = 10, p-value = 0.01412
```

11. Univariate GARCH models

Hide

```
spec_sp500 <- ugarchspec(
  mean.model     = arma_sp500$model,
  variance.model = list(model = "eGARCH",   garchOrder = c(1, 1)),
  distribution.model = "sstd")

spec_tb <- ugarchspec(
  mean.model     = arma_tb$model,
  variance.model = list(model = "sGARCH",   garchOrder = c(1, 1)),
  distribution.model = "std")

spec_gold <- ugarchspec(
  mean.model     = arma_gold$model,
  variance.model = list(model = "sGARCH",   garchOrder = c(1, 1)),
  distribution.model = "sged")

spec_oil <- ugarchspec(
  mean.model     = arma_oil$model,
  variance.model = list(model = "gjrGARCH", garchOrder = c(1, 1)),
  distribution.model = "sstd")

fit_sp500 <- ugarchfit(spec_sp500, data = r_sp500, solver = "hybrid")
fit_tb    <- ugarchfit(spec_tb,    data = r_tb,    solver = "hybrid")
fit_gold  <- ugarchfit(spec_gold,  data = r_gold,  solver = "hybrid")
fit_oil   <- ugarchfit(spec_oil,   data = r_oil,   solver = "hybrid")

show(fit_sp500); show(fit_tb); show(fit_gold); show(fit_oil)
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : eGARCH(1,1)
## Mean Model   : ARFIMA(1,0,1)
## Distribution : sstd
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error    t value Pr(>|t|)
## mu      0.000504    0.000180     2.8034 0.005056
## ar1     0.577915    0.067516     8.5597 0.000000
## ma1    -0.608872    0.066127    -9.2076 0.000000
## omega  -0.265775    0.005470   -48.5905 0.000000
## alpha1 -0.182809    0.020584    -8.8809 0.000000
## beta1   0.971764    0.000023 41570.3241 0.000000
## gamma1  0.069487    0.010196     6.8153 0.000000
## skew    0.800904    0.036726    21.8078 0.000000
## shape   8.825341    1.943290     4.5414 0.000006
##
## Robust Standard Errors:
##         Estimate  Std. Error    t value Pr(>|t|)
## mu      0.000504    0.000172     2.9344 0.003342
## ar1     0.577915    0.016587    34.8406 0.000000
## ma1    -0.608872    0.016656   -36.5556 0.000000
## omega  -0.265775    0.005320   -49.9559 0.000000
## alpha1 -0.182809    0.024734    -7.3909 0.000000
## beta1   0.971764    0.000024 40967.2865 0.000000
## gamma1  0.069487    0.009623     7.2213 0.000000
## skew    0.800904    0.038032    21.0590 0.000000
## shape   8.825341    2.103073     4.1964 0.000027
##
## LogLikelihood : 4135.978
##
## Information Criteria
## ------------------------------------
##
## Akaike       -6.5874
## Bayes        -6.5505
## Shibata      -6.5875
## Hannan-Quinn -6.5735
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.3077  0.5791
## Lag[2*(p+q)+(p+q)-1][5]    1.5556  0.9969
## Lag[4*(p+q)+(p+q)-1][9]    2.1610  0.9767
## d.o.f=2
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.2868  0.5923
## Lag[2*(p+q)+(p+q)-1][5]    1.7814  0.6709
## Lag[4*(p+q)+(p+q)-1][9]    3.6172  0.6542
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]    0.0711 0.500 2.000  0.7897
## ARCH Lag[5]    3.4653 1.440 1.667  0.2290
## ARCH Lag[7]    4.0899 2.315 1.543  0.3338
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  2.4129
## Individual Statistics:
## mu     0.1780
## ar1    0.2119
```

```
## ma1    0.2161
## omega  0.2939
## alpha1 0.2465
## beta1  0.2717
## gamma1 0.0793
## skew   0.2919
## shape  1.0632
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          2.1 2.32 2.82
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value    prob sig
## Sign Bias           1.7209 0.08551   *
## Negative Sign Bias  0.6312 0.52805
## Positive Sign Bias  0.5254 0.59938
## Joint Effect        3.2805 0.35036
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     25.29      0.15115
## 2    30     35.90      0.17654
## 3    40     49.76      0.11593
## 4    50     62.84      0.08839
##
##
## Elapsed time : 0.7280381
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,1)
## Mean Model   : ARFIMA(1,0,1)
## Distribution : std
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error   t value Pr(>|t|)
## mu     -0.000193    0.000225  -0.86070 0.389404
## ar1     0.618773    0.209227   2.95743 0.003102
## ma1    -0.667070    0.197847  -3.37165 0.000747
## omega   0.000000    0.000001   0.14715 0.883014
## alpha1  0.033807    0.005678   5.95407 0.000000
## beta1   0.965189    0.005827 165.63111 0.000000
## shape  27.966482   15.820693   1.76771 0.077108
##
## Robust Standard Errors:
##         Estimate  Std. Error   t value Pr(>|t|)
## mu     -0.000193    0.000217 -0.891496 0.372663
## ar1     0.618773    0.159086  3.889546 0.000100
## ma1    -0.667070    0.147289 -4.528998 0.000006
## omega   0.000000    0.000004  0.026712 0.978690
## alpha1  0.033807    0.042418  0.797000 0.425451
## beta1   0.965189    0.042802 22.549904 0.000000
## shape  27.966482   48.261543  0.579478 0.562267
##
## LogLikelihood : 4027.635
##
## Information Criteria
## ------------------------------------
##
## Akaike       -6.4176
## Bayes        -6.3889
## Shibata      -6.4177
## Hannan-Quinn -6.4068
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.1627  0.6867
## Lag[2*(p+q)+(p+q)-1][5]    1.5894  0.9960
## Lag[4*(p+q)+(p+q)-1][9]    2.3495  0.9643
## d.o.f=2
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      1.488  0.2225
## Lag[2*(p+q)+(p+q)-1][5]     3.689  0.2954
## Lag[4*(p+q)+(p+q)-1][9]     4.783  0.4620
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]    0.9083 0.500 2.000  0.3406
## ARCH Lag[5]    1.0625 1.440 1.667  0.7145
## ARCH Lag[7]    1.8842 2.315 1.543  0.7418
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  281.0658
## Individual Statistics:
## mu      0.18530
## ar1     0.10808
## ma1     0.09076
## omega  71.67841
## alpha1  0.34379
## beta1   0.35116
```

```
## shape   0.22129
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.69 1.9 2.35
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value    prob sig
## Sign Bias           1.9622 0.04996  **
## Negative Sign Bias  1.3341 0.18240
## Positive Sign Bias  0.6647 0.50634
## Joint Effect        4.0786 0.25310
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     25.64       0.1404
## 2    30     33.26       0.2672
## 3    40     48.80       0.1351
## 4    50     59.81       0.1386
##
##
## Elapsed time : 0.302238
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,1)
## Mean Model   : ARFIMA(1,0,1)
## Distribution : sged
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000488    0.000251   1.9483 0.051385
## ar1    -0.907565    0.074281 -12.2180 0.000000
## ma1     0.929430    0.066709  13.9325 0.000000
## omega   0.000007    0.000000  17.8708 0.000000
## alpha1  0.077941    0.008871   8.7863 0.000000
## beta1   0.850317    0.014934  56.9369 0.000000
## skew    0.955903    0.037027  25.8167 0.000000
## shape   1.407303    0.076861  18.3098 0.000000
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000488    0.000251   1.9433 0.051985
## ar1    -0.907565    0.080451 -11.2809 0.000000
## ma1     0.929430    0.068933  13.4832 0.000000
## omega   0.000007    0.000000  14.1251 0.000000
## alpha1  0.077941    0.005757  13.5374 0.000000
## beta1   0.850317    0.011429  74.3985 0.000000
## skew    0.955903    0.040518  23.5923 0.000000
## shape   1.407303    0.075908  18.5397 0.000000
##
## LogLikelihood : 4078.021
##
## Information Criteria
## ------------------------------------
##
## Akaike       -6.4964
## Bayes        -6.4637
## Shibata      -6.4965
## Hannan-Quinn -6.4841
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.1341  0.7143
## Lag[2*(p+q)+(p+q)-1][5]    1.1024  1.0000
## Lag[4*(p+q)+(p+q)-1][9]    1.9956  0.9847
## d.o.f=2
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.2617  0.6090
## Lag[2*(p+q)+(p+q)-1][5]    3.1019  0.3888
## Lag[4*(p+q)+(p+q)-1][9]    5.2511  0.3935
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]  0.004176 0.500 2.000  0.9485
## ARCH Lag[5]  4.682709 1.440 1.667  0.1216
## ARCH Lag[7]  5.590090 2.315 1.543  0.1713
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  62.2955
## Individual Statistics:
## mu     0.8172
## ar1    0.4269
## ma1    0.3320
## omega  4.1619
```

```
## alpha1 0.8896
## beta1  0.5255
## skew   0.1902
## shape  0.2526
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.89 2.11 2.59
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value    prob sig
## Sign Bias           0.8045 0.42127
## Negative Sign Bias  0.8264 0.40875
## Positive Sign Bias  1.8295 0.06757   *
## Joint Effect        4.8920 0.17988
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     10.10       0.9505
## 2    30     21.25       0.8500
## 3    40     31.12       0.8116
## 4    50     40.50       0.8012
##
##
## Elapsed time : 0.8592939
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : gjrGARCH(1,1)
## Mean Model   : ARFIMA(1,0,1)
## Distribution : sstd
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000841    0.000540  1.55740 0.119375
## ar1    -0.299919    0.810968 -0.36983 0.711510
## ma1     0.314463    0.806362  0.38998 0.696553
## omega   0.000027    0.000011  2.58703 0.009681
## alpha1  0.088181    0.028599  3.08337 0.002047
## beta1   0.835995    0.041330 20.22732 0.000000
## gamma1  0.024812    0.033700  0.73626 0.461570
## skew    0.830333    0.036212 22.92996 0.000000
## shape   9.076831    1.986366  4.56957 0.000005
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu      0.000841    0.000531  1.58221 0.113600
## ar1    -0.299919    0.349029 -0.85930 0.390177
## ma1     0.314463    0.348326  0.90278 0.366641
## omega   0.000027    0.000011  2.39651 0.016552
## alpha1  0.088181    0.041302  2.13502 0.032759
## beta1   0.835995    0.043930 19.03012 0.000000
## gamma1  0.024812    0.042780  0.58001 0.561910
## skew    0.830333    0.040718 20.39246 0.000000
## shape   9.076831    1.980429  4.58326 0.000005
##
## LogLikelihood : 3152.108
##
## Information Criteria
## ------------------------------------
##
## Akaike       -5.0169
## Bayes        -4.9801
## Shibata      -5.0170
## Hannan-Quinn -5.0031
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      0.218  0.6405
## Lag[2*(p+q)+(p+q)-1][5]     2.838  0.5754
## Lag[4*(p+q)+(p+q)-1][9]     4.495  0.5724
## d.o.f=2
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                      1.100  0.2942
## Lag[2*(p+q)+(p+q)-1][5]     3.331  0.3499
## Lag[4*(p+q)+(p+q)-1][9]     5.444  0.3672
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]     2.772 0.500 2.000 0.09595
## ARCH Lag[5]     3.953 1.440 1.667 0.17817
## ARCH Lag[7]     5.048 2.315 1.543 0.21976
##
## Nyblom stability test
## ------------------------------------
## Joint Statistic:  2.5663
## Individual Statistics:
## mu     0.7307
## ar1    0.1552
```

```
## ma1    0.1594
## omega  0.1933
## alpha1 0.2662
## beta1  0.2343
## gamma1 0.4470
## skew   0.5952
## shape  0.1955
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          2.1 2.32 2.82
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value   prob sig
## Sign Bias           0.5181 0.6045
## Negative Sign Bias  1.3600 0.1741
## Positive Sign Bias  0.9740 0.3303
## Joint Effect        3.2251 0.3582
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     18.68       0.4773
## 2    30     26.90       0.5773
## 3    40     32.08       0.7761
## 4    50     41.37       0.7722
##
##
## Elapsed time : 0.8891199
```

12. Standardised residual diagnostics

Hide

```
resid_list <- list(
  r_sp500 = residuals(fit_sp500, standardize = TRUE),
  r_tb    = residuals(fit_tb,    standardize = TRUE),
  r_gold  = residuals(fit_gold,  standardize = TRUE),
  r_oil   = residuals(fit_oil,   standardize = TRUE)
)
invisible(lapply(names(resid_list), function(nm) {
  r <- resid_list[[nm]]
  cat(sprintf("\n=== %s ===\n", nm))
  cat("Ljung-Box (resid):\n");   print(Box.test(r,   lag = 10, type = "Ljung-Box"))
  cat("Ljung-Box (resid^2):\n"); print(Box.test(r^2, lag = 10, type = "Ljung-Box"))
  cat("ARCH test:\n");           print(ArchTest(r^2))
}))
```

```
##
## === r_sp500 ===
## Ljung-Box (resid):
##
##  Box-Ljung test
##
## data:  r
## X-squared = 3.8632, df = 10, p-value = 0.9533
##
## Ljung-Box (resid^2):
##
##  Box-Ljung test
##
## data:  r^2
## X-squared = 6.7166, df = 10, p-value = 0.7519
##
## ARCH test:
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  r^2
## Chi-squared = 3.9086, df = 12, p-value = 0.985
##
##
## === r_tb ===
## Ljung-Box (resid):
##
##  Box-Ljung test
##
## data:  r
## X-squared = 4.1909, df = 10, p-value = 0.9383
##
## Ljung-Box (resid^2):
##
##  Box-Ljung test
##
## data:  r^2
## X-squared = 6.6393, df = 10, p-value = 0.759
##
## ARCH test:
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  r^2
## Chi-squared = 7.6073, df = 12, p-value = 0.815
##
##
## === r_gold ===
## Ljung-Box (resid):
##
##  Box-Ljung test
##
## data:  r
## X-squared = 3.7417, df = 10, p-value = 0.9582
##
## Ljung-Box (resid^2):
##
##  Box-Ljung test
##
## data:  r^2
## X-squared = 9.1932, df = 10, p-value = 0.5139
##
## ARCH test:
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  r^2
## Chi-squared = 10.871, df = 12, p-value = 0.54
##
##
## === r_oil ===
## Ljung-Box (resid):
##
##  Box-Ljung test
##
## data:  r
```

```
## X-squared = 11.833, df = 10, p-value = 0.2964
##
## Ljung-Box (resid^2):
##
##  Box-Ljung test
##
## data:  r^2
## X-squared = 9.4258, df = 10, p-value = 0.4922
##
## ARCH test:
##
##  ARCH LM-test; Null hypothesis: no ARCH effects
##
## data:  r^2
## Chi-squared = 0.80004, df = 12, p-value = 1
```

13. DCC-GARCH

Hide

```
spec_dcc <- dccspec(
  uspec        = multispec(list(spec_sp500, spec_tb, spec_gold, spec_oil)),
  dccOrder     = c(1, 1),
  distribution = "mvt"
)

dcc_model <- dccfit(spec_dcc, data = returns, out.sample = 250)
dcc_model
```

```
##
## *---------------------------------*
## *          DCC GARCH Fit          *
## *---------------------------------*
##
## Distribution         :  mvt
## Model                :  DCC(1,1)
## No. Parameters       :  42
## [VAR GARCH DCC UncQ] : [0+33+3+6]
## No. Series           :  4
## No. Obs.             :  1003
## Log-Likelihood       :  12431.25
## Av.Log-Likelihood    :  12.39
##
## Optimal Parameters
## ------------------------------------
##                   Estimate  Std. Error     t value Pr(>|t|)
## [r_sp500].mu      0.000417    0.000244     1.70752 0.087725
## [r_sp500].ar1    -0.517071    0.027969   -18.48712 0.000000
## [r_sp500].ma1     0.545353    0.028133    19.38465 0.000000
## [r_sp500].omega  -0.252951    0.006369   -39.71475 0.000000
## [r_sp500].alpha1 -0.177986    0.022235    -8.00471 0.000000
## [r_sp500].beta1   0.972956    0.000036 26947.85368 0.000000
## [r_sp500].gamma1  0.068627    0.021498     3.19219 0.001412
## [r_sp500].skew    0.827809    0.046528    17.79157 0.000000
## [r_sp500].shape  11.908146    4.061635     2.93186 0.003369
## [r_tb].mu        -0.000421    0.000282    -1.49655 0.134511
## [r_tb].ar1        0.501729    0.158653     3.16243 0.001565
## [r_tb].ma1       -0.562658    0.153245    -3.67162 0.000241
## [r_tb].omega      0.000001    0.000001     1.09851 0.271984
## [r_tb].alpha1     0.034747    0.008194     4.24079 0.000022
## [r_tb].beta1      0.953398    0.010703    89.07852 0.000000
## [r_tb].shape     42.377967   33.831040     1.25264 0.210338
## [r_gold].mu       0.000189    0.000269     0.70149 0.482997
## [r_gold].ar1     -0.843652    0.020663   -40.82906 0.000000
## [r_gold].ma1      0.887055    0.018378    48.26622 0.000000
## [r_gold].omega    0.000002    0.000001     3.05163 0.002276
## [r_gold].alpha1   0.026513    0.002398    11.05765 0.000000
## [r_gold].beta1    0.946723    0.003687   256.78976 0.000000
## [r_gold].skew     0.969805    0.042823    22.64691 0.000000
## [r_gold].shape    1.378156    0.086156    15.99610 0.000000
## [r_oil].mu        0.001097    0.000601     1.82583 0.067875
## [r_oil].ar1      -0.369637    0.108085    -3.41986 0.000627
## [r_oil].ma1       0.394482    0.104394     3.77878 0.000158
## [r_oil].omega     0.000020    0.000008     2.42044 0.015502
## [r_oil].alpha1    0.072558    0.028424     2.55267 0.010690
## [r_oil].beta1     0.857989    0.032140    26.69547 0.000000
## [r_oil].gamma1    0.048526    0.036208     1.34020 0.180181
## [r_oil].skew      0.792138    0.043315    18.28776 0.000000
## [r_oil].shape     9.809163    2.603597     3.76754 0.000165
## [Joint]dcca1      0.023671    0.004823     4.90809 0.000001
## [Joint]dccb1      0.940092    0.012544    74.94386 0.000000
## [Joint]mshape    10.689206    1.248996     8.55824 0.000000
##
## Information Criteria
## ---------------------
##
## Akaike       -24.704
## Bayes        -24.499
## Shibata      -24.708
## Hannan-Quinn -24.626
##
##
## Elapsed time : 8.508042
```

Hide

```
print(DCCtest(residuals(dcc_model)))
```

```
## $H0
## [1] "Constant Probability"
##
## $p.value
## [1] 0.1809875
##
## $statistic
## [1] 3.418655
```

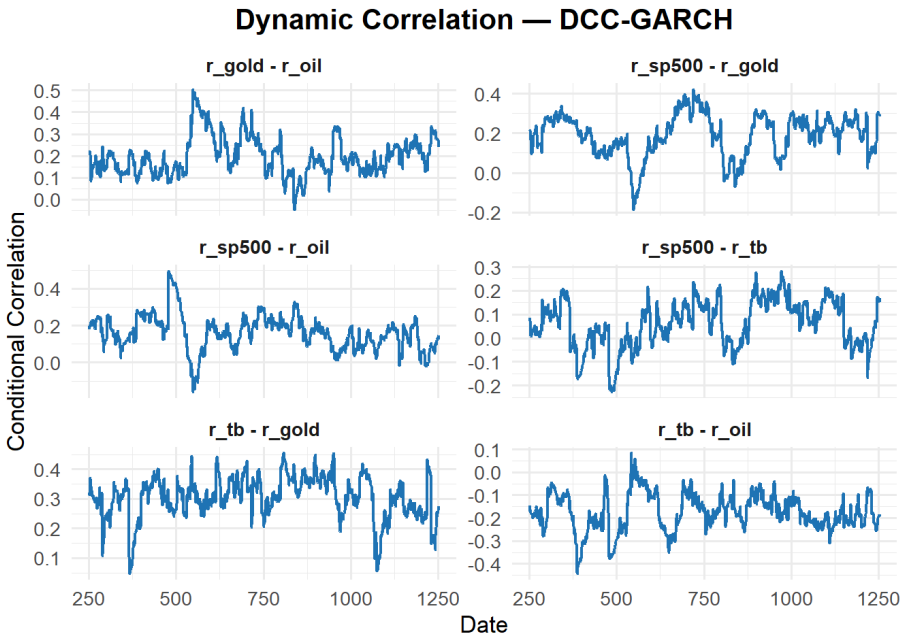
14. Dynamic correlation & covariance plots

Hide

```
plot_dynamic_corr <- function(model, title_label) {
  R_t         <- rcor(model)
  T_in        <- dim(R_t)[3]
  dates_in    <- if (!is.null(index(returns))) tail(index(returns), T_in) else 1:T_in
  asset_names <- colnames(returns)
  corr_list   <- list()
  for (i in 1:(ncol(returns) - 1))
    for (j in (i + 1):ncol(returns))
      corr_list[[length(corr_list) + 1]] <- data.frame(
        Date = dates_in, Corr = R_t[i, j, ],
        Pair = paste(asset_names[i], "-", asset_names[j]))
  ggplot(do.call(rbind, corr_list), aes(x = Date, y = Corr)) +
    geom_line(color = "#1f77b4", linewidth = 0.8) +
    facet_wrap(~ Pair, ncol = 2, scales = "free_y") +
    labs(title = paste("Dynamic Correlation —", title_label),
         x = "Date", y = "Conditional Correlation") +
    theme_minimal(base_size = 13) +
    theme(strip.text = element_text(face = "bold"),
          plot.title = element_text(face = "bold", hjust = 0.5))
}

plot_dynamic_cov <- function(model, title_label) {
  H_t         <- rcov(model)
  T_in        <- dim(H_t)[3]
  dates_in    <- if (!is.null(index(returns))) tail(index(returns), T_in) else 1:T_in
  asset_names <- colnames(returns)
  cov_list    <- list()
  for (i in 1:(ncol(returns) - 1))
    for (j in (i + 1):ncol(returns))
      cov_list[[length(cov_list) + 1]] <- data.frame(
        Date = dates_in, Cov = H_t[i, j, ],
        Pair = paste(asset_names[i], "-", asset_names[j]))
  ggplot(do.call(rbind, cov_list), aes(x = Date, y = Cov)) +
    geom_line(color = "#D62728", linewidth = 0.8) +
    facet_wrap(~ Pair, ncol = 2, scales = "free_y") +
    labs(title = paste("Conditional Covariance —", title_label),
         x = "Date", y = "Conditional Covariance") +
    theme_minimal(base_size = 13) +
    theme(strip.text = element_text(face = "bold"),
          plot.title = element_text(face = "bold", hjust = 0.5))
}
```

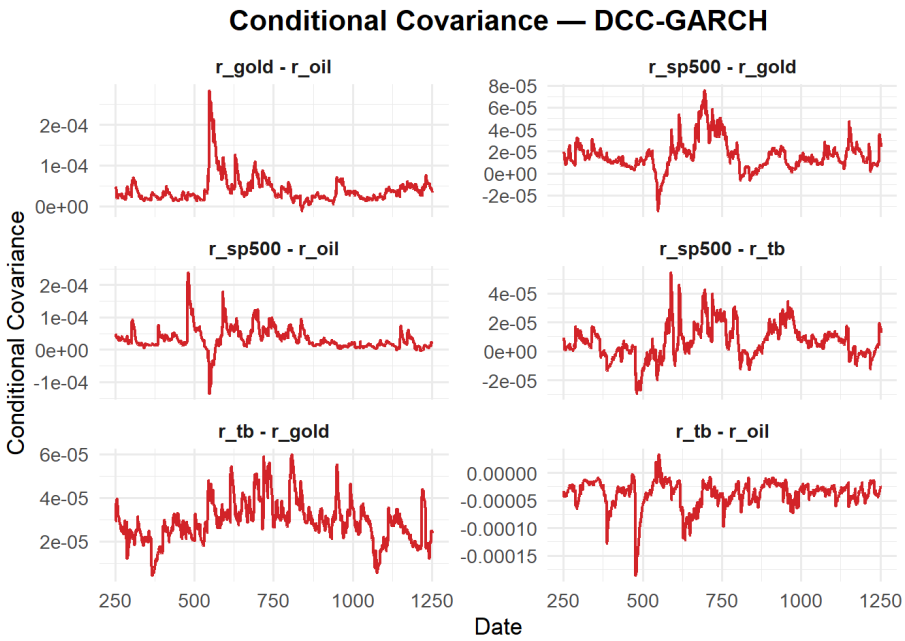
Hide

```
plot_dynamic_corr(dcc_model, "DCC-GARCH")
```



Hide

```
plot_dynamic_cov(dcc_model,  "DCC-GARCH")
```



15. GO-GARCH

Hide

```
set.seed(123)
spec_go <- gogarchspec(
  mean.model     = list(model = "constant", robust = FALSE),
  variance.model = list(model = "eGARCH", garchOrder = c(1, 1)),
  distribution.model = "magh",
  ica = "fastica"
)
gogarch_model <- gogarchfit(spec = spec_go, data = returns, out.sample = 250)
show(gogarch_model)
```

```
##
## *------------------------------*
## *        GO-GARCH Fit          *
## *------------------------------*
##
## Mean Model       : CONSTANT
## GARCH Model      : eGARCH
## Distribution : magh
## ICA Method       : fastica
## No. Factors      : 4
## No. Periods      : 1003
## Log-Likelihood   : 12391.78
## ------------------------------------
##
## U (rotation matrix) :
##
##        [,1]    [,2]    [,3]    [,4]
## [1,] -0.711 -0.0617 -0.6889  0.1278
## [2,]  0.376  0.0485 -0.2255  0.8976
## [3,]  0.577  0.1360 -0.6861 -0.4213
## [4,] -0.142  0.9876  0.0625  0.0219
##
## A (mixing matrix) :
##
##          [,1]      [,2]     [,3]     [,4]
## [1,]  0.00504  0.000284 -0.00371 -0.00828
## [2,] -0.00649 -0.004760  0.00314 -0.00626
## [3,] -0.00218  0.005994  0.00415 -0.00486
## [4,]  0.01477  0.000249  0.01534 -0.00204
```

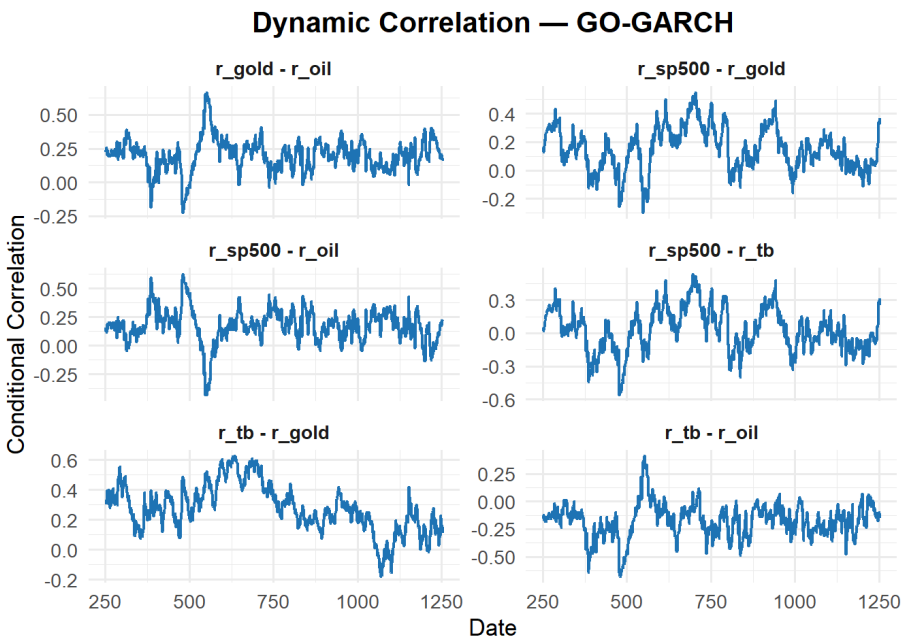
Hide

```
gogarch_model@mfit$garchcoef
```

```
##                 [,1]         [,2]         [,3]         [,4]
## omega    -0.006370078 -0.001050555 -0.005051287 -0.001219817
## alpha1   -0.076403865  0.047801716  0.017249874  0.068863259
## beta1     0.945044879  0.980987668  0.970381493  0.988925169
## gamma1    0.152748655  0.001361324  0.176277904  0.044570224
## skew     -0.250840871 -0.698680561 -0.120137694  0.167233438
## shape     2.727863611  0.250057033  2.369410249  3.343993874
## ghlambda -1.230050566 -4.639061347 -2.498546282  3.392243631
```

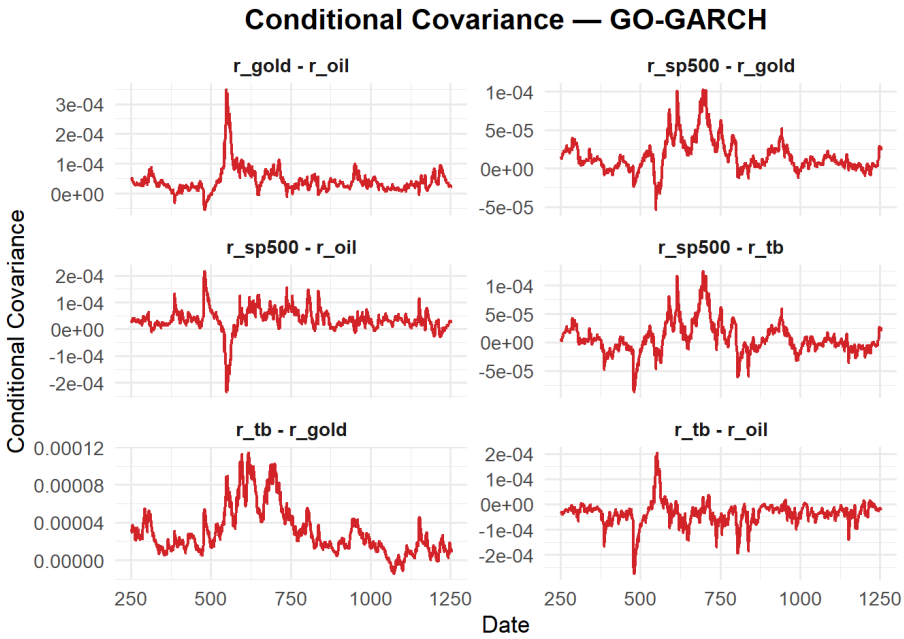
Hide

```
plot_dynamic_corr(gogarch_model, "GO-GARCH")
```



Hide

```
plot_dynamic_cov(gogarch_model,  "GO-GARCH")
```



16. Forecast evaluation — variance & correlation

Hide

```
n           <- nrow(returns)
asset_names <- colnames(returns)

# Realised variance: expanding window — uses only data up to day t (no lookahead)
var_expand <- lapply(seq_along(asset_names), function(j) {
  v <- rep(NA, n)
  for (t in 2:n) v[t] <- var(returns[1:t, j], na.rm = TRUE)
  v
})
names(var_expand) <- asset_names

real_var   <- lapply(asset_names, function(nm) tail(var_expand[[nm]], 250))
names(real_var) <- asset_names
real_sigma <- lapply(asset_names, function(nm) sqrt(real_var[[nm]]))
names(real_sigma) <- asset_names
```

Hide

```
dcc_forecast       <- dccforecast(dcc_model, n.ahead = 1, n.roll = 249)
dcc_cov_forecast   <- rcov(dcc_forecast)
dcc_cor_forecast   <- rcor(dcc_forecast)
dcc_sigma_forecast <- sigma(dcc_forecast)
dcc_mean_forecast  <- fitted(dcc_forecast)

go_forecast       <- gogarchforecast(gogarch_model, n.ahead = 1, n.roll = 249)
go_cov_forecast   <- rcov(go_forecast)
go_cor_forecast   <- rcor(go_forecast)
go_sigma_forecast <- sigma(go_forecast)
go_mean_forecast  <- fitted(go_forecast)
```

Hide

```
# DCC forecasted variance
n_days_dcc  <- dim(dcc_sigma_forecast)[3]
dcc_var_mat <- matrix(NA, n_days_dcc, 4, dimnames = list(NULL, asset_names))
for (t in 1:n_days_dcc) dcc_var_mat[t, ] <- dcc_sigma_forecast[1,, t]^2

# GO forecasted variance
n_days_go  <- length(go_cov_forecast)
go_var_mat <- matrix(NA, n_days_go, 4, dimnames = list(NULL, asset_names))
for (t in 1:n_days_go) go_var_mat[t, ] <- diag(go_cov_forecast[[t]][,, 1])
```

Hide

```
eval_df <- data.frame(
  Asset         = asset_names,
  DCC_MSE_Var   = sapply(asset_names, function(nm) mse(real_var[[nm]], dcc_var_mat[, nm])),
  DCC_MAE_Var   = sapply(asset_names, function(nm) mae(real_var[[nm]], dcc_var_mat[, nm])),
  DCC_MSE_Sigma = sapply(asset_names, function(nm) mse(real_sigma[[nm]], sqrt(dcc_var_mat[, nm]))),
  DCC_MAE_Sigma = sapply(asset_names, function(nm) mae(real_sigma[[nm]], sqrt(dcc_var_mat[, nm]))),
  GO_MSE_Var    = sapply(asset_names, function(nm) mse(real_var[[nm]], go_var_mat[, nm])),
  GO_MAE_Var    = sapply(asset_names, function(nm) mae(real_var[[nm]], go_var_mat[, nm])),
  GO_MSE_Sigma  = sapply(asset_names, function(nm) mse(real_sigma[[nm]], sqrt(go_var_mat[, nm]))),
  GO_MAE_Sigma  = sapply(asset_names, function(nm) mae(real_sigma[[nm]], sqrt(go_var_mat[, nm])))
)
kable(eval_df, digits = 8, caption = "MSE/MAE: Forecasted Variance & Volatility") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed"), full_width = FALSE)
```

MSE/MAE: Forecasted Variance & Volatility

	Asset	DCC_MSE_Var	DCC_MAE_Var	DCC_MSE_Sigma	DCC_MAE_Sigma	GO_MSE_Var	GO_MAE_Var	GO_MSE_Sigma	GO_MAE_Sigma
r_sp500	r_sp500	2e-08	0.00008074	2.060e-05	0.00358798	0e+00	0.00004391	6.250e-06	0.00220675
r_tb	r_tb	0e+00	0.00003525	4.280e-06	0.00190082	0e+00	0.00002589	2.730e-06	0.00117240
r_gold	r_gold	0e+00	0.00003369	5.220e-06	0.00149818	0e+00	0.00001667	1.250e-06	0.00088164
r_oil	r_oil	5e-08	0.00017156	2.419e-05	0.00403445	3e-08	0.00012962	1.347e-05	0.00306224

Hide

```
# Dynamic correlation evaluation
cov_expand <- vector("list", n)
for (t in 2:n) cov_expand[[t]] <- cov(returns[1:t, ], use = "pairwise.complete.obs")
cor_expand     <- lapply(cov_expand, function(x) if (!is.null(x)) cov2cor(x) else NULL)
pairs_list     <- combn(asset_names, 2, simplify = FALSE)
cor_expand_oos <- cor_expand[(n - 249):n]

real_cor_series <- lapply(pairs_list, function(pair)
  sapply(cor_expand_oos, function(x) if (!is.null(x)) x[pair[1], pair[2]] else NA))
dcc_cor_series  <- lapply(pairs_list, function(pair)
  sapply(dcc_cor_forecast, function(x) x[pair[1], pair[2], 1]))
go_cor_series   <- lapply(pairs_list, function(pair)
  sapply(go_cor_forecast, function(x) x[pair[1], pair[2], 1]))
```

Hide

```
cor_eval <- data.frame(
  Pair    = sapply(pairs_list, paste, collapse = "-"),
  DCC_MAE = mapply(function(r, f) mean(abs(r - f), na.rm = TRUE), real_cor_series, dcc_cor_series),
  DCC_MSE = mapply(function(r, f) mean((r - f)^2,  na.rm = TRUE), real_cor_series, dcc_cor_series),
  GO_MAE  = mapply(function(r, f) mean(abs(r - f), na.rm = TRUE), real_cor_series, go_cor_series),
  GO_MSE  = mapply(function(r, f) mean((r - f)^2,  na.rm = TRUE), real_cor_series, go_cor_series)
)
kable(cor_eval, digits = 8, caption = "MSE/MAE: Forecasted Dynamic Correlation") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed"), full_width = FALSE)
```

MSE/MAE: Forecasted Dynamic Correlation

Pair	DCC_MAE	DCC_MSE	GO_MAE	GO_MSE
r_sp500-r_tb	0.06373636	0.00682296	0.16759614	0.04840525
r_sp500-r_gold	0.07753052	0.00945437	0.09190446	0.01562535
r_sp500-r_oil	0.11181100	0.01963514	0.10980271	0.02062319
r_tb-r_gold	0.07468181	0.00828824	0.20162160	0.05545387
r_tb-r_oil	0.05171468	0.00422938	0.11811056	0.02154404
r_gold-r_oil	0.07006245	0.00782075	0.09030500	0.01312072

17. Core VaR / ES / Backtest functions

Hide

```
# Fissler-Ziegel (2016) joint loss function for VaR & ES forecasts.
FZ <- function(r, VaR, ES, alpha) {
  I     <- as.numeric(r < VaR)
  term1 <- (I - alpha) * (VaR - r)
  term2 <- (1 / alpha) * I * (ES - r)
  term1 + term2
}

# Parametric VaR & ES: Normal + Student-t
compute_var_es_param <- function(mu_vec, sigma_vec, alpha, df_t = 5) {
  z_n <- qnorm(alpha)
  t_a <- qt(alpha, df = df_t)
  list(
    VaR_N = mu_vec + z_n * sigma_vec,
    ES_N  = mu_vec - (dnorm(z_n) / alpha) * sigma_vec,
    VaR_T = mu_vec + t_a * sigma_vec,
    ES_T  = mu_vec - sigma_vec * ((df_t + t_a^2) / (df_t - 1)) * (dt(t_a, df_t) / alpha)
  )
}
```

Hide

```
# Dynamic Quantile (DQ) Test — Engle & Manganelli (2004)
# Tests H0: hit sequence is uncorrelated with its own lags and VaR
# hit = I(r_t < VaR_t) - alpha
# Regress hit_t on const, hit_{t-1}, ..., hit_{t-p}, VaR_t
dq_test <- function(actual, VaR, alpha, p = 4) {
  T_    <- length(actual)
  hit   <- as.numeric(actual < VaR) - alpha
  # build regressor matrix (no lookahead: VaR is 1-step-ahead forecast)
  if (T_ <= p + 1) return(list(stat = NA_real_, pval = NA_real_, df = NA_integer_))
  Y  <- hit[(p + 1):T_]
  X  <- cbind(1,
              sapply(1:p, function(k) hit[(p + 1 - k):(T_ - k)]),
              VaR[(p + 1):T_])
  colnames(X) <- c("const", paste0("hit_lag", 1:p), "VaR")
  tryCatch({
    fit  <- lm(Y ~ X - 1)
    n    <- length(Y)
    k    <- ncol(X)
    RSS  <- sum(residuals(fit)^2)
    # Wald statistic: beta' (X'X)^{-1} beta / (RSS/n) * n
    # Under H0 DQ ~ chi-sq(k)
    b    <- coef(fit)
    V    <- (RSS / n) * solve(t(X) %*% X)
    stat <- as.numeric(t(b) %*% solve(V) %*% b)
    pval <- pchisq(stat, df = k, lower.tail = FALSE)
    list(stat = round(stat, 4), pval = round(pval, 4), df = k)
  }, error = function(e) list(stat = NA_real_, pval = NA_real_, df = NA_integer_))
}
```

Hide

```
# run_backtest: UC, CC (Christoffersen 1998), DQ, ES, FZ
run_backtest <- function(actual, VaR, ES, alpha, model_label, dist_label) {

  # UC & CC (rugarch)
  # VaRTest returns a flat list: $uc.LRp, $cc.LRp  (scalars)
  uc_p <- NA_real_; cc_p <- NA_real_
  tryCatch({
    vt <- VaRTest(alpha, actual = actual, VaR = VaR, conf.level = 1 - alpha)
    if (!is.null(vt[["uc.LRp"]]) && length(vt[["uc.LRp"]]) == 1L)
      uc_p <- round(as.numeric(vt[["uc.LRp"]]), 4)
    if (!is.null(vt[["cc.LRp"]]) && length(vt[["cc.LRp"]]) == 1L)
      cc_p <- round(as.numeric(vt[["cc.LRp"]]), 4)
  }, error = function(e) NULL, warning = function(w) NULL)

  # ESTest
  es_p <- NA_real_
  tryCatch({
    et <- ESTest(alpha, actual = actual, VaR = VaR, ES = ES,
                 conf.level = 1 - alpha, boot = FALSE)
    if (!is.null(et[["p.value"]]) && length(et[["p.value"]]) >= 1L)
      es_p <- round(as.numeric(et[["p.value"]][1]), 4)
  }, error = function(e) NULL, warning = function(w) NULL)

  # DQ test
  dq    <- dq_test(actual, VaR, alpha)
  dq_p  <- if (!is.null(dq$pval)) dq$pval else NA_real_

  n_viol <- sum(actual < VaR, na.rm = TRUE)

  data.frame(
    Model       = model_label, Dist  = dist_label, Alpha = alpha,
    Mean_VaR    = round(mean(VaR, na.rm = TRUE), 6),
    Mean_ES     = round(mean(ES,  na.rm = TRUE), 6),
    N_Violation = as.integer(n_viol),
    Exc_Rate    = round(n_viol / length(actual), 4),
    UC_pval     = uc_p,
    CC_pval     = cc_p,
    DQ_pval     = dq_p,
    ES_pval     = es_p,
    FZ_Score    = round(mean(FZ(actual, VaR, ES, alpha), na.rm = TRUE), 8),
    stringsAsFactors = FALSE
  )
}

backtest_multi_alpha <- function(actual, mu_vec, sigma_vec, alphas,
                                 model_label, df_t = 5) {
  do.call(rbind, lapply(alphas, function(a) {
    res <- compute_var_es_param(mu_vec, sigma_vec, a, df_t)
    rbind(
      run_backtest(actual, res$VaR_N, res$ES_N, a, model_label, "Normal"),
      run_backtest(actual, res$VaR_T, res$ES_T, a, model_label, "Student-t")
    )
  }))
}
```

18. Optimized Portfolio

This section introduces three core functions used in the portfolio optimization process for constructing the Tangency Portfolio, which maximizes the Sharpe ratio and represents the optimal trade-off between expected return and risk. Geometrically, the Tangency Portfolio is the point where the efficient frontier is tangent to the Capital Allocation Line (CAL) drawn from the risk-free rate.

The compute_tangency_weights function calculates the portfolio weights that maximize the Sharpe ratio under a no short-selling constraint using quadratic programming. If no asset is expected to outperform the risk-free rate, the function automatically switches to the Minimum Variance Portfolio, ensuring that the optimizer always produces a valid solution.

Hide

```
shrink_mean <- function(R_window) {
  colMeans(R_window)
}

# Compute Tangency weights (no short-selling).
# Fallback to MVP if all excess returns <= 0 or QP fails.
compute_tangency_weights <- function(mu_vec, Sigma, rf = 0) {
  N      <- length(mu_vec)
  excess <- mu_vec - rf

  mvp_solve <- function() {
    w <- quadprog::solve.QP(
      Dmat = 2 * Sigma + diag(1e-8, N),
      dvec = rep(0, N),
      Amat = cbind(rep(1, N), diag(N)),
      bvec = c(1, rep(0, N)), meq = 1
    )$solution
    pmax(w, 0) / sum(pmax(w, 0))
  }

  if (all(excess <= 1e-10)) return(mvp_solve())

  tryCatch({
    sol   <- quadprog::solve.QP(
      Dmat = 2 * Sigma + diag(1e-8, N),
      dvec = rep(0, N),
      Amat = cbind(excess, diag(N)),
      bvec = c(1, rep(0, N)), meq = 1
    )
    w_raw <- pmax(sol$solution, 0)
    if (sum(w_raw) < 1e-10) return(mvp_solve())
    w_raw / sum(w_raw)
  }, error = function(e) mvp_solve())
}
```

The compute_tangency_rolling function updates portfolio weights daily in an out-of-sample framework, using only information available at the time. Expected returns and covariance matrices are taken from one-step-ahead forecasts (from DCC-GARCH or GO-GARCH), while the risk-free rate is the most recently observed T-bill rate. This strict timing ensures that the backtest remains free from lookahead bias.

Hide

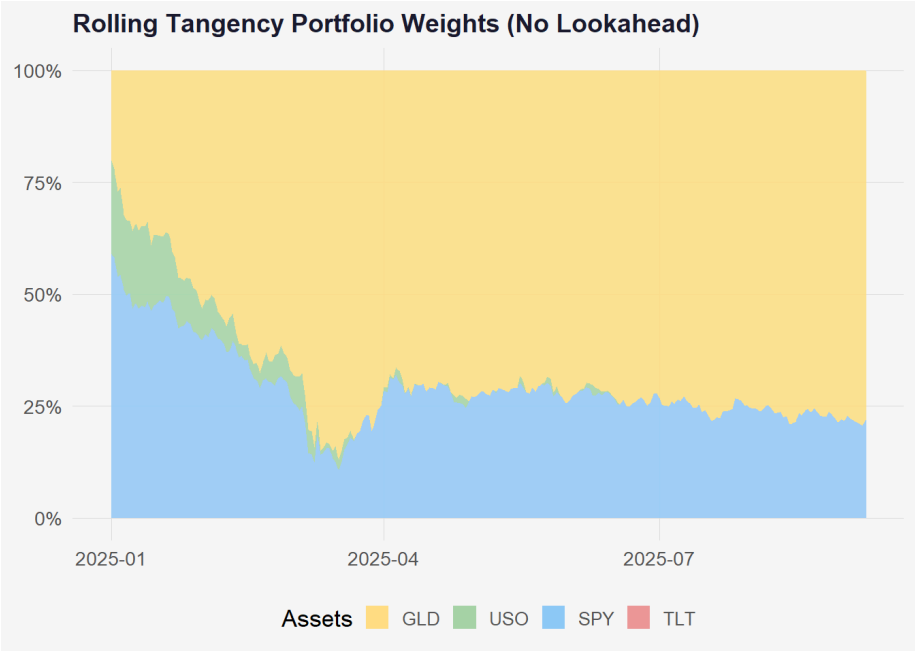
```
# Rolling Tangency weights from multivariate model forecasts.
# STRICT no-lookahead: weight at step t uses ONLY
#   - mu from mean_fc_arr[1,,t]  (1-step-ahead forecast made at t-1)
#   - Sigma from cov_fc_list[[t]] (1-step-ahead forecast made at t-1)
# rf_vec_oos: optional vector of daily rf rates aligned to OOS period (length = n_ahead).
# If NULL, scalar rf_daily is used for all steps.
compute_tangency_rolling <- function(mean_fc_arr, cov_fc_list,
                                     rf_daily = 0, rf_vec_oos = NULL) {
  n_ahead  <- length(cov_fc_list)
  n_assets <- dim(cov_fc_list[[1]])[1]
  W        <- matrix(NA, nrow = n_ahead, ncol = n_assets)
  colnames(W) <- colnames(returns)

  for (t in 1:n_ahead) {
    mu_t    <- as.numeric(mean_fc_arr[1, , t])
    Sigma_t <- cov_fc_list[[t]][,, 1]
    # Use time-varying rf if provided (no lookahead: rf at t uses rate from t-1)
    rf_t    <- if (!is.null(rf_vec_oos) && length(rf_vec_oos) >= t)
                 rf_vec_oos[t] else rf_daily
    if (is.na(rf_t)) rf_t <- rf_daily
    W[t, ]  <- compute_tangency_weights(mu_t, Sigma_t, rf = rf_t)
  }
  W
}
```

Hide

```
T_total   <- nrow(returns)
N         <- ncol(returns)
window_in <- 1003
n_out     <- 250
start_out <- T_total - n_out + 1
alphas <- c(0.01, 0.025, 0.05)
df_t   <- 5

# Storage
W_tan      <- matrix(NA, nrow = n_out, ncol = N,
                          dimnames = list(NULL, asset_names))
sigma_para_roll <- numeric(n_out)
mu_para_roll    <- numeric(n_out)
r_p_para        <- numeric(n_out)

for (t in 1:n_out) {
  day_OOS   <- start_out + t - 1
  idx_end   <- day_OOS - 1              # strictly before forecast day
  idx_start <- idx_end - window_in + 1
  R_window  <- returns[idx_start:idx_end, ]

  #NO lookahead
  mu_shrink        <- shrink_mean(R_window)
  Sigma_t          <- cov(R_window)

  # rf at this step: T-bill rate observed on day_OOS - 1 (no lookahead)
  rf_t             <- rf_vec[idx_end]
  if (is.na(rf_t)) rf_t <- rf_is_mean

  # Tangency weights decided at t-1, applied to day t return
  w_t              <- compute_tangency_weights(mu_shrink, Sigma_t, rf = rf_t)
  W_tan[t, ]  <- w_t
  mu_para_roll[t]   <- sum(w_t * mu_shrink)
  sigma_para_roll[t] <- sqrt(t(w_t) %*% Sigma_t %*% w_t)
  r_p_para[t]       <- as.numeric(w_t %*% returns[day_OOS, ])  # realised
}
```

Hide

```
library(tidyr)
df <- as.data.frame(W_tan)
df$date <- seq.Date(from = as.Date("2025-01-01"), by = "day", length.out = nrow(df))

df_long <- pivot_longer(df,
  cols = c(r_sp500, r_tb, r_gold, r_oil),
  names_to  = "group",
  values_to = "value"
)

ggplot(df_long, aes(x = date, y = value, fill = group)) +
  geom_area(position = "stack", alpha = 0.85) +
  scale_y_continuous(labels = scales::percent) +
  scale_x_date(date_labels = "%Y-%m") +
  scale_fill_manual(
    values = c(
      "r_sp500" = "#90CAF9",
      "r_tb"    = "#EF9A9A",
      "r_gold"  = "#FFE082",
      "r_oil"   = "#A5D6A7"
    ),
    labels = c(
      "r_sp500" = "SPY",
      "r_tb"    = "TLT",
      "r_gold"  = "GLD",
      "r_oil"   = "USO"
    )
  ) +
  guides(fill = guide_legend(nrow = 1, override.aes = list(alpha = 1))) +
  labs(x = NULL, y = NULL, fill = "Assets",
       title = "Rolling Tangency Portfolio Weights (No Lookahead)") +
  theme_minimal(base_size = 13) +
  theme(
    plot.title       = element_text(face = "bold", size = 14, color = "#1a1a2e"),
    panel.grid.minor = element_blank(),
    panel.grid.major = element_line(color = "#e0e0e0", linewidth = 0.3),
    legend.position  = "bottom",
    legend.direction = "horizontal",
    legend.text      = element_text(size = 10, color = "#555555"),
    legend.key.size  = unit(0.5, "cm"),
    legend.spacing.x = unit(0.3, "cm"),
    axis.text        = element_text(color = "#555555"),
    plot.background  = element_rect(fill = "#f9f9f9", color = NA),
    panel.background = element_rect(fill = "#f9f9f9", color = NA)
  )
```



#19. Parametric VaR & ES backtest (alpha = 1%, 2.5%, 5%)

Hide

```
para_results <- backtest_multi_alpha(r_p_para, mu_para_roll, sigma_para_roll,
                                     alphas, "Parametric", df_t)

kable(para_results, caption = "Parametric Backtest — multiple alpha levels") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed"), full_width = FALSE)
```

Parametric Backtest — multiple alpha levels

Model	Dist	Alpha	Mean_VaR	Mean_ES	N_Violation	Exc_Rate	UC_pval	CC_pval	DQ_pval	ES_pval	FZ_Score
Parametric	Normal	0.010	-0.017851	-0.020537	6	0.024	0.0594	0.0029	0.0000	0.0313	0.0233341
Parametric	Student-t	0.010	-0.026082	-0.034701	3	0.012	0.7580	0.9194	0.9681	0.1903	0.0062508
Parametric	Normal	0.025	-0.014947	-0.017942	11	0.044	0.0817	0.0467	0.1221	0.0478	0.0109835
Parametric	Student-t	0.025	-0.019787	-0.027324	4	0.016	0.3296	0.0798	0.0059	0.0665	0.0061695
Parametric	Normal	0.050	-0.012450	-0.015762	19	0.076	0.0787	0.1908	0.0846	0.0508	0.0069450
Parametric	Student-t	0.050	-0.015384	-0.022319	9	0.036	0.2860	0.0562	0.0165	0.2030	0.0035846

20. DCC-GARCH — VaR & ES (alpha = 1%, 2.5%, 5%)

Hide

```
# r_p_para  : realised OOS returns of the portfolio
# W_tan_para: rolling weights (no-lookahead, James-Stein + rolling cov)
returns_oos <- tail(returns, 250)

# DCC: project DCC covariance through W_tan_para weights -> portfolio sigma
# mu: James-Stein shrinkage mean from Parametric rolling (consistent across models)
n_fc_dcc  <- length(dcc_cov_forecast)
mu_dcc    <- numeric(n_fc_dcc)
sigma_dcc <- numeric(n_fc_dcc)

for (t in 1:n_fc_dcc) {
  # Weight from Parametric (no-lookahead) — SAME portfolio for all models
  w_t          <- W_tan[t, ]
  # DCC 1-step-ahead covariance forecast (made at t-1)
  Sigma_t      <- dcc_cov_forecast[[t]][,, 1]
  # James-Stein mean from Parametric rolling window (already in mu_para_roll)
  mu_dcc[t]    <- mu_para_roll[t]
  # Portfolio sigma from DCC covariance + shared weights
  sigma_dcc[t] <- sqrt(as.numeric(t(w_t) %*% Sigma_t %*% w_t))
}
```

Hide

```
# Shared portfolio return — same r_p_para used for all models
dcc_results <- backtest_multi_alpha(r_p_para, mu_dcc, sigma_dcc, alphas, "DCC-GARCH", df_t)

kable(dcc_results, caption = "DCC-GARCH VaR/ES Backtest") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed"), full_width = FALSE)
```

DCC-GARCH VaR/ES Backtest

Model	Dist	Alpha	Mean_VaR	Mean_ES	N_Violation	Exc_Rate	UC_pval	CC_pval	DQ_pval	ES_pval	FZ_Score
DCC-GARCH	Normal	0.010	-0.019299	-0.022195	4	0.016	0.3805	0.0873	0.0074	0.0030	0.0214266
DCC-GARCH	Student-t	0.010	-0.028176	-0.037472	3	0.012	0.7580	0.9194	0.9912	0.4918	0.0005169
DCC-GARCH	Normal	0.025	-0.016167	-0.019397	6	0.024	0.9188	0.2962	0.3505	0.0160	0.0105980
DCC-GARCH	Student-t	0.025	-0.021387	-0.029515	4	0.016	0.3296	0.0798	0.0058	0.0990	0.0043766
DCC-GARCH	Normal	0.050	-0.013474	-0.017046	12	0.048	0.8839	0.2837	0.4906	0.0544	0.0061320
DCC-GARCH	Student-t	0.050	-0.016638	-0.024118	6	0.024	0.0366	0.0335	0.0338	0.1007	0.0039718

21. GO-GARCH — VaR & ES (alpha = 1%, 2.5%, 5%)

Hide

```
# GO-GARCH: same shared portfolio (W_tan_para), only Sigma_t from GO-GARCH
n_fc_go  <- length(go_cov_forecast)
mu_go    <- numeric(n_fc_go)
sigma_go <- numeric(n_fc_go)

for (t in 1:n_fc_go) {
  w_t         <- W_tan[t, ]
  Sigma_t     <- go_cov_forecast[[t]][,, 1]
  mu_go[t]    <- mu_para_roll[t]
  sigma_go[t] <- sqrt(as.numeric(t(w_t) %*% Sigma_t %*% w_t))
}
```

Hide

```
# Shared portfolio return
go_results <- backtest_multi_alpha(r_p_para, mu_go, sigma_go, alphas, "GO-GARCH", df_t)

kable(go_results, caption = "GO-GARCH VaR/ES Backtest") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed"), full_width = FALSE)
```

GO-GARCH VaR/ES Backtest

Model	Dist	Alpha	Mean_VaR	Mean_ES	N_Violation	Exc_Rate	UC_pval	CC_pval	DQ_pval	ES_pval	FZ_Score
GO-GARCH	Normal	0.010	-0.016446	-0.018927	6	0.024	0.0594	0.0503	0.0132	0.0185	0.0261433
GO-GARCH	Student-t	0.010	-0.024049	-0.032011	3	0.012	0.7580	0.9194	0.9936	0.1622	0.0080668
GO-GARCH	Normal	0.025	-0.013763	-0.016530	13	0.052	0.0167	0.0211	0.2169	0.0412	0.0119072
GO-GARCH	Student-t	0.025	-0.018234	-0.025196	5	0.020	0.6000	0.1801	0.0007	0.1189	0.0059987
GO-GARCH	Normal	0.050	-0.011457	-0.014516	21	0.084	0.0240	0.0769	0.1039	0.0466	0.0073096
GO-GARCH	Student-t	0.050	-0.014167	-0.020573	12	0.048	0.8839	0.2837	0.4399	0.2680	0.0030958

22. Historical Simulation — VaR & ES

Hide

```
window_hs <- 250

hs_results <- do.call(rbind, lapply(alphas, function(a) {
  VaR_hs <- numeric(n_out)
  ES_hs  <- numeric(n_out)

  for (t in 1:n_out) {
    day_OOS     <- start_out + t - 1
    idx_end     <- day_OOS - 1
    idx_start   <- idx_end - window_hs + 1
    # Shared portfolio weights — same W_tan
    w_t         <- W_tan[t, ]
    port_window <- as.numeric(returns[idx_start:idx_end, ] %*% w_t)

    VaR_hs[t] <- quantile(port_window, probs = a, type = 7)
    below      <- port_window[port_window <= VaR_hs[t]]
    ES_hs[t]  <- if (length(below) > 0) mean(below) else VaR_hs[t]
  }
  # r_p_para is the shared portfolio return — consistent with other models
  run_backtest(r_p_para, VaR_hs, ES_hs, a, "Historical Simulation", "Empirical")
}))

kable(hs_results, caption = "Historical Simulation Backtest") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed"), full_width = FALSE)
```

Historical Simulation Backtest

Model	Dist	Alpha	Mean_VaR	Mean_ES	N_Violation	Exc_Rate	UC_pval	CC_pval	DQ_pval	ES_pval	FZ_Score
Historical Simulation	Empirical	0.010	-0.020140	-0.026620	4	0.016	0.3805	0.0873	0.0036	0.0647	0.0152197
Historical Simulation	Empirical	0.025	-0.016195	-0.021395	8	0.032	0.4965	0.0486	0.0021	0.1389	0.0070227
Historical Simulation	Empirical	0.050	-0.013549	-0.018356	14	0.056	0.6691	0.4225	0.6430	0.1545	0.0045270

23. Summary backtest table (by alpha)

Hide

```
all_results <- rbind(dcc_results, go_results, para_results, hs_results)

flag <- function(p) ifelse(is.na(p), "N/A", ifelse(p >= 0.05, "Pass", "Fail"))
all_results$UC_flag <- flag(all_results$UC_pval)
all_results$CC_flag <- flag(all_results$CC_pval)
all_results$DQ_flag <- flag(all_results$DQ_pval)
all_results$ES_flag <- flag(all_results$ES_pval)

tbls <- lapply(alphas, function(a) {
  sub     <- all_results[all_results$Alpha == a, ]
  display <- sub[, c("Model","Dist","Mean_VaR","Mean_ES","N_Violation","Exc_Rate",
                     "UC_pval","UC_flag","CC_pval","CC_flag",
                     "DQ_pval","DQ_flag","ES_pval","ES_flag","FZ_Score")]
  colnames(display) <- c("Model","Dist","Mean VaR","Mean ES","Violations","Exc Rate",
                         "p-UC","UC","p-CC","CC","p-DQ","DQ","p-ES","ES","FZ Score")

  kable(display, digits = 6, format = "html",
        caption = sprintf("Backtest Summary — Alpha = %.1f%%", a * 100)) %>%
    kable_styling(bootstrap_options = c("striped","hover","condensed","bordered"),
                  full_width = FALSE, font_size = 11) %>%
    htmltools::HTML()
})

htmltools::tagList(tbls)
```

Backtest Summary — Alpha = 1.0%

	Model	Dist	Mean VaR	Mean ES	Violations	Exc Rate	p-UC	UC	p-CC	CC	p-DQ	DQ	p-ES	ES	FZ Score
1	DCC-GARCH	Normal	-0.019299	-0.022195	4	0.016	0.3805	Pass	0.0873	Pass	0.0074	Fail	0.0030	Fail	0.021427
2	DCC-GARCH	Student-t	-0.028176	-0.037472	3	0.012	0.7580	Pass	0.9194	Pass	0.9912	Pass	0.4918	Pass	0.000517
7	GO-GARCH	Normal	-0.016446	-0.018927	6	0.024	0.0594	Pass	0.0503	Pass	0.0132	Fail	0.0185	Fail	0.026143
8	GO-GARCH	Student-t	-0.024049	-0.032011	3	0.012	0.7580	Pass	0.9194	Pass	0.9936	Pass	0.1622	Pass	0.008067
13	Parametric	Normal	-0.017851	-0.020537	6	0.024	0.0594	Pass	0.0029	Fail	0.0000	Fail	0.0313	Fail	0.023334
14	Parametric	Student-t	-0.026082	-0.034701	3	0.012	0.7580	Pass	0.9194	Pass	0.9681	Pass	0.1903	Pass	0.006251
19	Historical Simulation	Empirical	-0.020140	-0.026620	4	0.016	0.3805	Pass	0.0873	Pass	0.0036	Fail	0.0647	Pass	0.015220

Backtest Summary — Alpha = 2.5%

	Model	Dist	Mean VaR	Mean ES	Violations	Exc Rate	p-UC	UC	p-CC	CC	p-DQ	DQ	p-ES	ES	FZ Score
3	DCC-GARCH	Normal	-0.016167	-0.019397	6	0.024	0.9188	Pass	0.2962	Pass	0.3505	Pass	0.0160	Fail	0.010598
4	DCC-GARCH	Student-t	-0.021387	-0.029515	4	0.016	0.3296	Pass	0.0798	Pass	0.0058	Fail	0.0990	Pass	0.004377
9	GO-GARCH	Normal	-0.013763	-0.016530	13	0.052	0.0167	Fail	0.0211	Fail	0.2169	Pass	0.0412	Fail	0.011907
10	GO-GARCH	Student-t	-0.018234	-0.025196	5	0.020	0.6000	Pass	0.1801	Pass	0.0007	Fail	0.1189	Pass	0.005999
15	Parametric	Normal	-0.014947	-0.017942	11	0.044	0.0817	Pass	0.0467	Fail	0.1221	Pass	0.0478	Fail	0.010983
16	Parametric	Student-t	-0.019787	-0.027324	4	0.016	0.3296	Pass	0.0798	Pass	0.0059	Fail	0.0665	Pass	0.006169
20	Historical Simulation	Empirical	-0.016195	-0.021395	8	0.032	0.4965	Pass	0.0486	Fail	0.0021	Fail	0.1389	Pass	0.007023

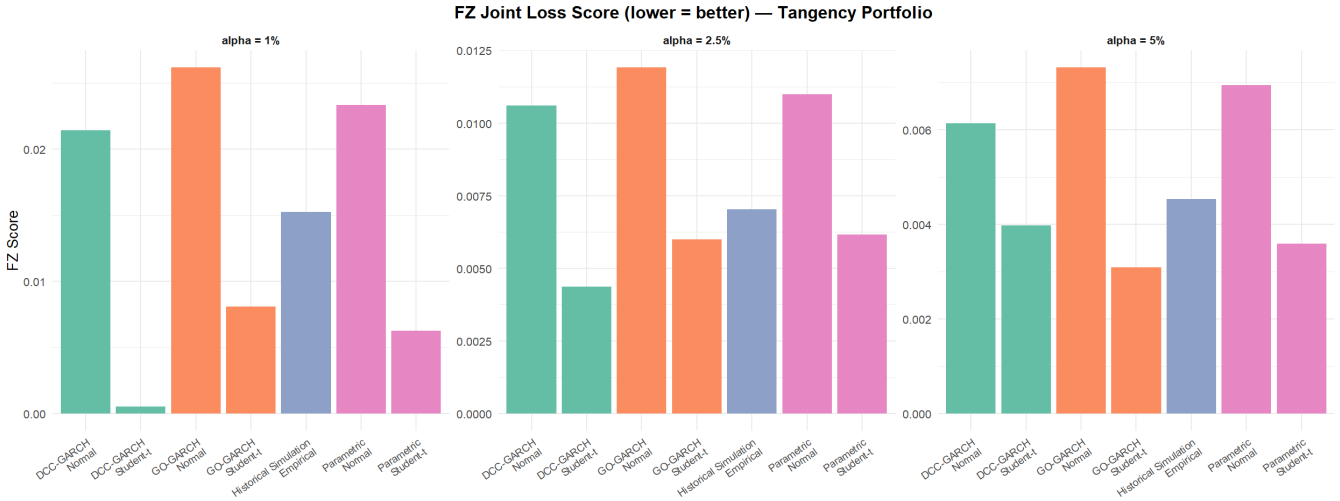
Backtest Summary — Alpha = 5.0%

	Model	Dist	Mean VaR	Mean ES	Violations	Exc Rate	p-UC	UC	p-CC	CC	p-DQ	DQ	p-ES	ES	FZ Score
5	DCC-GARCH	Normal	-0.013474	-0.017046	12	0.048	0.8839	Pass	0.2837	Pass	0.4906	Pass	0.0544	Pass	0.006132
6	DCC-GARCH	Student-t	-0.016638	-0.024118	6	0.024	0.0366	Fail	0.0335	Fail	0.0338	Fail	0.1007	Pass	0.003972
11	GO-GARCH	Normal	-0.011457	-0.014516	21	0.084	0.0240	Fail	0.0769	Pass	0.1039	Pass	0.0466	Fail	0.007310
12	GO-GARCH	Student-t	-0.014167	-0.020573	12	0.048	0.8839	Pass	0.2837	Pass	0.4399	Pass	0.2680	Pass	0.003096
17	Parametric	Normal	-0.012450	-0.015762	19	0.076	0.0787	Pass	0.1908	Pass	0.0846	Pass	0.0508	Pass	0.006945
18	Parametric	Student-t	-0.015384	-0.022319	9	0.036	0.2860	Pass	0.0562	Pass	0.0165	Fail	0.2030	Pass	0.003585
21	Historical Simulation	Empirical	-0.013549	-0.018356	14	0.056	0.6691	Pass	0.4225	Pass	0.6430	Pass	0.1545	Pass	0.004527

24. FZ Score & Violation rate plots

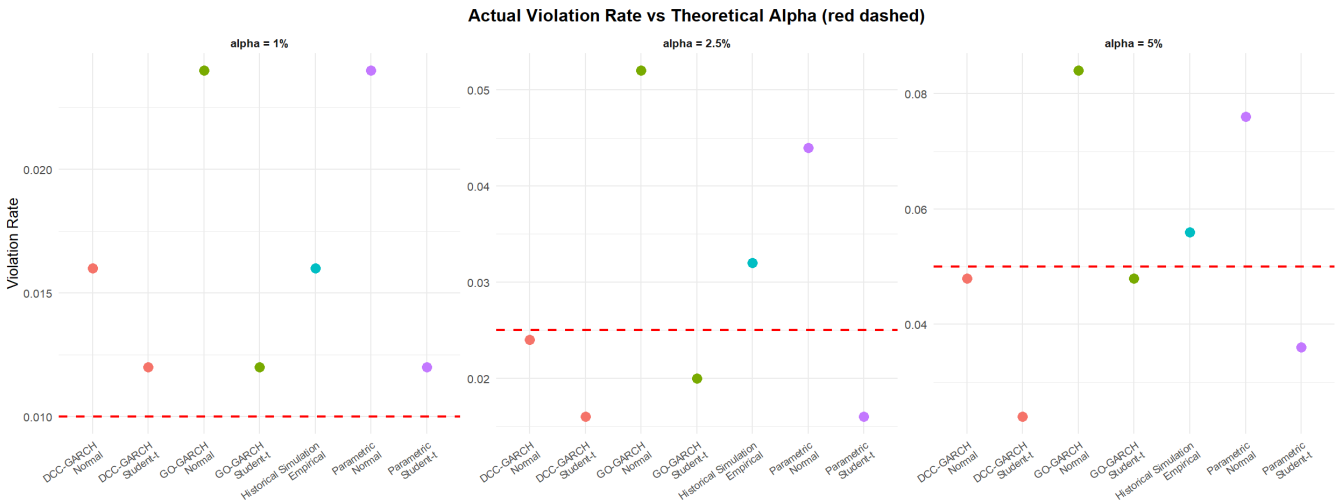
Hide

```
all_results$Label       <- paste(all_results$Model, all_results$Dist, sep = "\n")
all_results$Alpha_label <- paste0("alpha = ", all_results$Alpha * 100, "%")

p1 <- ggplot(all_results, aes(x = Label, y = FZ_Score, fill = Model)) +
  geom_col(show.legend = FALSE) +
  facet_wrap(~ Alpha_label, scales = "free_y") +
  labs(title = "FZ Joint Loss Score (lower = better) — Tangency Portfolio",
       x = NULL, y = "FZ Score") +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 35, hjust = 1, size = 7),
        plot.title  = element_text(face = "bold", hjust = 0.5),
        strip.text  = element_text(face = "bold")) +
  scale_fill_brewer(palette = "Set2")
print(p1)
```



Hide

```
alpha_ref <- data.frame(Alpha_label = paste0("alpha = ", alphas * 100, "%"), Alpha = alphas)
p2 <- ggplot(all_results, aes(x = Label, y = Exc_Rate, color = Model)) +
  geom_point(size = 3, show.legend = FALSE) +
  geom_hline(data = alpha_ref, aes(yintercept = Alpha),
             linetype = "dashed", color = "red", linewidth = 0.8) +
  facet_wrap(~ Alpha_label, scales = "free_y") +
  labs(title = "Actual Violation Rate vs Theoretical Alpha (red dashed)",
       x = NULL, y = "Violation Rate") +
  theme_minimal(base_size = 10) +
  theme(axis.text.x = element_text(angle = 35, hjust = 1, size = 7),
        plot.title  = element_text(face = "bold", hjust = 0.5),
        strip.text  = element_text(face = "bold"))
print(p2)
```



25. VaR/ES Ranking (averaged over all alpha levels)

Hide

```
ranking_var <- all_results %>%
  group_by(Model, Dist) %>%
  summarise(
    FZ_Mean      = round(mean(FZ_Score, na.rm = TRUE), 8),
    ExcRate_Mean = round(mean(Exc_Rate, na.rm = TRUE), 4),
    UC_Pass_pct  = round(mean(UC_pval >= 0.05, na.rm = TRUE) * 100, 1),
    CC_Pass_pct  = round(mean(CC_pval >= 0.05, na.rm = TRUE) * 100, 1),
    DQ_Pass_pct  = round(mean(DQ_pval >= 0.05, na.rm = TRUE) * 100, 1),
    ES_Pass_pct  = round(mean(ES_pval >= 0.05, na.rm = TRUE) * 100, 1),
    .groups = "drop"
  ) %>%
  arrange(FZ_Mean)

colnames(ranking_var) <- c("Model","Dist","FZ Mean","Exc Rate Mean",
                           "% Pass UC","% Pass CC","% Pass DQ","% Pass ES")
kable(ranking_var, caption = "VaR/ES Ranking — Averaged over all alpha levels") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed","bordered"),
                full_width = FALSE, font_size = 13)
```

VaR/ES Ranking — Averaged over all alpha levels

Model	Dist	FZ Mean	Exc Rate Mean	% Pass UC	% Pass CC	% Pass DQ	% Pass ES
DCC-GARCH	Student-t	0.0029551	0.0173	66.7	66.7	33.3	100.0
Parametric	Student-t	0.0053349	0.0213	100.0	100.0	33.3	100.0
GO-GARCH	Student-t	0.0057204	0.0267	100.0	100.0	66.7	100.0
Historical Simulation	Empirical	0.0089232	0.0347	100.0	66.7	33.3	100.0
DCC-GARCH	Normal	0.0127189	0.0293	100.0	100.0	66.7	33.3
Parametric	Normal	0.0137542	0.0480	100.0	33.3	66.7	33.3
GO-GARCH	Normal	0.0151200	0.0533	33.3	66.7	66.7	0.0

26. Diebold-Mariano test for FZ loss differences

Tests whether the difference in FZ scores between model pairs is statistically significant (H0: equal predictive accuracy).

Hide

```
dm_alpha <- 0.01
df_stress <- 5

get_fz_series <- function(actual, VaR, ES, alpha) FZ(actual, VaR, ES, alpha)

make_var_es_both <- function(mu_vec, sigma_vec, alpha, df_t = df_stress) {
  res <- compute_var_es_param(mu_vec, sigma_vec, alpha, df_t)
  list(
    VaR_N = res$VaR_N, ES_N = res$ES_N,
    VaR_T = res$VaR_T, ES_T = res$ES_T
  )
}

res_dcc <- make_var_es_both(mu_dcc,  sigma_dcc, dm_alpha)
res_go  <- make_var_es_both(mu_go,   sigma_go,  dm_alpha)
res_para <- make_var_es_both(mu_para_roll, sigma_para_roll, dm_alpha)

VaR_hs <- numeric(n_out)
ES_hs  <- numeric(n_out)

for (t in 1:n_out) {
  day_OOS        <- start_out + t - 1
  idx_end        <- day_OOS - 1
  idx_start      <- idx_end - 250 + 1
  w_t            <- W_tan[t, ]
  port_w         <- as.numeric(returns[idx_start:idx_end, ] %*% w_t)
  VaR_hs[t]     <- quantile(port_w, probs = dm_alpha, type = 7)
  ES_hs[t]      <- mean(port_w[port_w <= VaR_hs[t]])
}

# FZ loss series — Normal distribution
fz_dcc_N  <- get_fz_series(r_p_para, res_dcc$VaR_N,  res_dcc$ES_N,  dm_alpha)
fz_go_N   <- get_fz_series(r_p_para, res_go$VaR_N,   res_go$ES_N,   dm_alpha)
fz_para_N <- get_fz_series(r_p_para, res_para$VaR_N, res_para$ES_N, dm_alpha)

# FZ loss series — Student-t df=5
fz_dcc_T  <- get_fz_series(r_p_para, res_dcc$VaR_T,  res_dcc$ES_T,  dm_alpha)
fz_go_T   <- get_fz_series(r_p_para, res_go$VaR_T,   res_go$ES_T,   dm_alpha)
fz_para_T <- get_fz_series(r_p_para, res_para$VaR_T, res_para$ES_T, dm_alpha)

# HAC-robust DM test (Newey-West, automatic bandwidth)
dm_hac <- function(loss1, loss2, label1, label2) {
  d      <- loss1 - loss2
  T_     <- length(d)
  lag_nw <- floor(T_^(1/3))
  lm_d   <- lm(d ~ 1)
  V_hac  <- sandwich::NeweyWest(lm_d, lag = lag_nw, prewhite = FALSE)
  stat   <- mean(d) / sqrt(V_hac[1, 1] / T_)
  pval   <- 2 * pnorm(-abs(stat))
  data.frame(
    Model_1   = label1,
    Model_2   = label2,
    Mean_diff = round(mean(d), 8),
    DM_stat   = round(stat, 4),
    p_value   = round(pval, 4),
    Result    = ifelse(pval < 0.01,
                       ifelse(mean(d) < 0,
                              paste(label1, "better"),
                              paste(label2, "better")),
                       "No significant difference"),
    stringsAsFactors = FALSE
  )
}

dm_normal <- rbind(
  dm_hac(fz_dcc_N,  fz_go_N,   "DCC-GARCH",  "GO-GARCH"),
  dm_hac(fz_dcc_N,  fz_para_N, "DCC-GARCH",  "Parametric"),
  dm_hac(fz_go_N,   fz_para_N, "GO-GARCH",   "Parametric")
)

dm_t5 <- rbind(
  dm_hac(fz_dcc_T,  fz_go_T,   "DCC-GARCH",  "GO-GARCH"),
  dm_hac(fz_dcc_T,  fz_para_T, "DCC-GARCH",  "Parametric"),
  dm_hac(fz_go_T,   fz_para_T, "GO-GARCH",   "Parametric")
)
```

```
kable(dm_normal,
      caption = "Diebold-Mariano Test (HAC-robust) — FZ Loss at alpha=1%, Normal Distribution") %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed", "bordered"),
    full_width = FALSE, font_size = 12
  ) %>%
  footnote(
    general       = paste0(
      "H0: equal predictive accuracy. ",
      "Positive Mean_diff => Model_1 has higher FZ loss (worse). "
    ),
    general_title = "Note: "
  )
```

Diebold-Mariano Test (HAC-robust) — FZ Loss at alpha=1%, Normal Distribution

Model_1	Model_2	Mean_diff	DM_stat	p_value	Result
DCC-GARCH	GO-GARCH	-0.0047167	-23.7913	0	DCC-GARCH better
DCC-GARCH	Parametric	-0.0019075	-12.1375	0	DCC-GARCH better
GO-GARCH	Parametric	0.0028092	26.3802	0	Parametric better

Note:
H0: equal predictive accuracy. Positive Mean_diff => Model_1 has higher FZ loss (worse).

Hide

```
# Hiển thị kết quả — Student-t df=5
kable(dm_t5,
      caption = "Diebold-Mariano Test (HAC-robust) — FZ Loss at alpha=1%, Student-t Distribution (df=5)") %>%
  kable_styling(
    bootstrap_options = c("striped", "hover", "condensed", "bordered"),
    full_width = FALSE, font_size = 12
  ) %>%
  footnote(
    general       = paste0(
      "H0: equal predictive accuracy. ",
      "Positive Mean_diff => Model_1 has higher FZ loss (worse). ",
      "Student-t df=5: heavier tails than Normal — more conservative VaR/ES."
    ),
    general_title = "Note: "
  )
```

Diebold-Mariano Test (HAC-robust) — FZ Loss at alpha=1%, Student-t Distribution (df=5)

Model_1	Model_2	Mean_diff	DM_stat	p_value	Result
DCC-GARCH	GO-GARCH	-0.0075499	-24.3793	0	DCC-GARCH better
DCC-GARCH	Parametric	-0.0057339	-21.6789	0	DCC-GARCH better
GO-GARCH	Parametric	0.0018160	12.0457	0	Parametric better

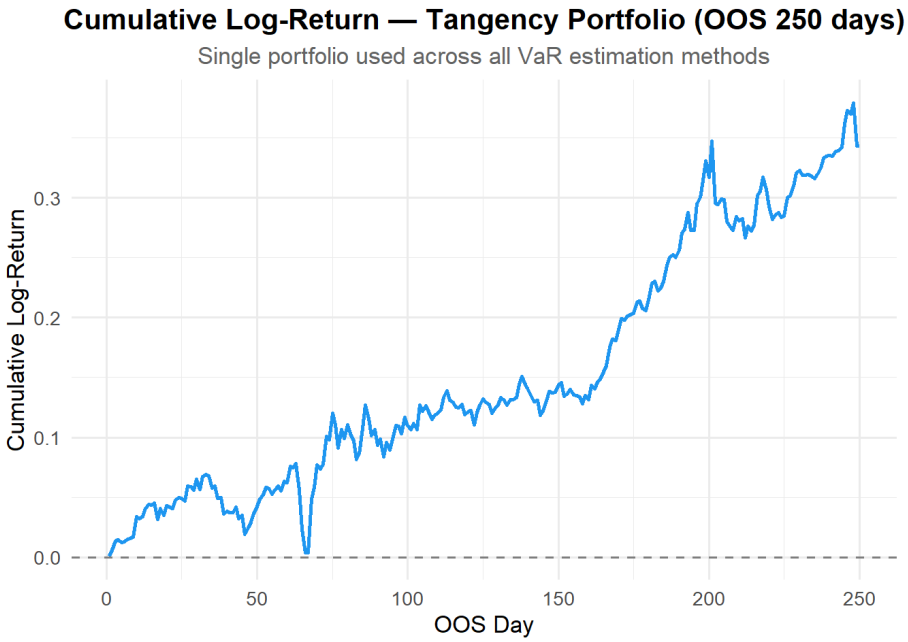
Note:
H0: equal predictive accuracy. Positive Mean_diff => Model_1 has higher FZ loss (worse). Student-t df=5: heavier tails than Normal — more conservative VaR/ES.

27. Portfolio performance analysis

27.1. Cumulative returns (OOS)

Hide

```
ggplot(data.frame(t = 1:250, CumReturn = cumsum(r_p_para)), aes(x = t, y = CumReturn)) +
  geom_line(linewidth = 1, color = "#2196F3") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "grey50") +
  labs(title    = "Cumulative Log-Return — Tangency Portfolio (OOS 250 days)",
       subtitle = "Single portfolio used across all VaR estimation methods",
       x = "OOS Day", y = "Cumulative Log-Return") +
  theme_minimal(base_size = 13) +
  theme(plot.title    = element_text(face = "bold", hjust = 0.5),
        plot.subtitle = element_text(hjust = 0.5, color = "grey40"))
```



27.2. Performance metrics table

Hide

```
calc_performance <- function(r, label, rf = rf_oos_mean, ann = 252) {
  r        <- na.omit(r)
  excess_r <- r - rf
  mu_d     <- mean(r)
  sd_d     <- sd(r)
  mu_ann   <- mu_d * ann
  sd_ann   <- sd_d * sqrt(ann)
  sharpe   <- mean(excess_r) / sd_d * sqrt(ann)

  # Sortino: downside deviation below rf
  downside <- r[r < rf]
  dd_ann   <- if (length(downside) > 1) sd(downside) * sqrt(ann) else NA_real_
  sortino  <- (mu_ann - rf * ann) / dd_ann

  # Max drawdown
  cum_r  <- cumsum(r)
  peak   <- cummax(cum_r)
  max_dd <- min(cum_r - peak)
  calmar <- if (abs(max_dd) > 0) mu_ann / abs(max_dd) else NA_real_

  # Win/Loss
  win_rate <- mean(r > 0) * 100
  avg_gain <- mean(r[r > 0])
  avg_loss <- mean(r[r < 0])
  gl_ratio <- abs(avg_gain / avg_loss)

  data.frame(
    Portfolio       = label,
    Total_Return    = round(sum(r), 6),
    Ann_Return      = round(mu_ann, 6),
    Ann_Volatility  = round(sd_ann, 6),
    Sharpe          = round(sharpe, 4),
    Sortino         = round(sortino, 4),
    Max_Drawdown    = round(max_dd, 6),
    Calmar          = round(calmar, 4),
    Win_Rate_pct    = round(win_rate, 2),
    GL_Ratio        = round(gl_ratio, 4),
    VaR_1pct_emp    = round(quantile(r, 0.01), 6),
    VaR_5pct_emp    = round(quantile(r, 0.05), 6),
    stringsAsFactors = FALSE
  )
}
```

Hide

```
perf_table <- calc_performance(r_p_para, "Tangency Portfolio")

perf_t        <- as.data.frame(t(perf_table[, -1]))
colnames(perf_t) <- perf_table$Portfolio
perf_t$Metric <- rownames(perf_t)
perf_t        <- perf_t[, c("Metric", "Tangency Portfolio")]
```

Hide

```
kable(perf_t, row.names = FALSE,
      caption = "Performance Metrics — Tangency Portfolio (250-day OOS)") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed","bordered"),
                full_width = FALSE, font_size = 12) %>%
  footnote(general = paste0(
    "Ann_Return & Ann_Volatility: annualised (x252). ",
    "Max_Drawdown: based on cumulative log-return. ",
    "rf_oos_mean (daily) = ", round(rf_oos_mean, 8),
    " (ann. ", round(rf_oos_mean*252*100, 4), "%). Source: FRED DTB3."),
    general_title = "Note: ")
```

Performance Metrics — Tangency Portfolio (250-day OOS)

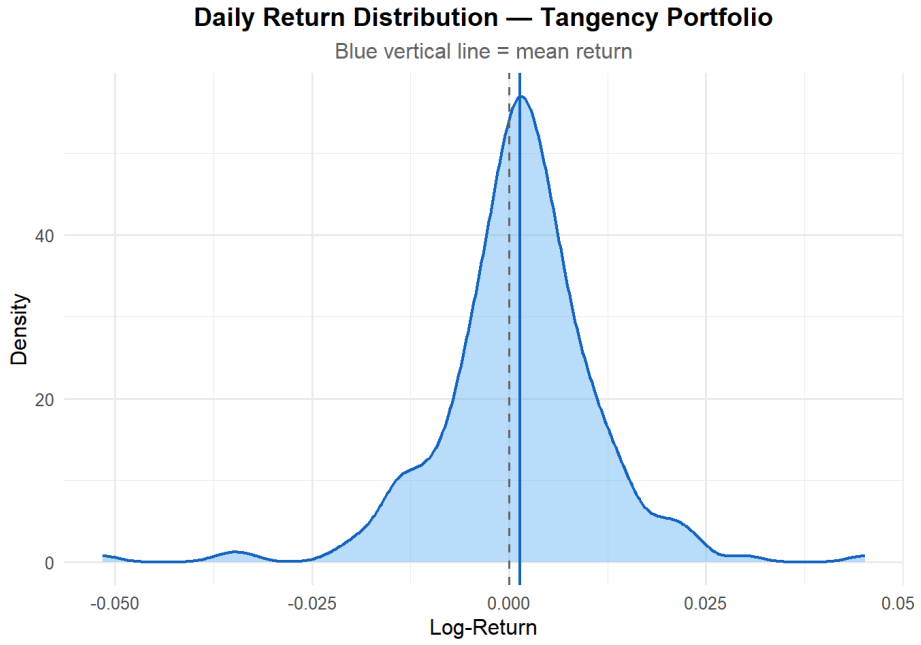
Metric	Tangency Portfolio
Total_Return	0.343533
Ann_Return	0.346281
Ann_Volatility	0.158723
Sharpe	2.109200
Sortino	2.640800
Max_Drawdown	-0.080921
Calmar	4.279300
Win_Rate_pct	60.400000
GL_Ratio	0.971600
VaR_1pct_emp	-0.027861
VaR_5pct_emp	-0.014037

Note:
Ann_Return & Ann_Volatility: annualised (x252). Max_Drawdown: based on cumulative log-return. rf_oos_mean (daily) = 4.567e-05 (ann. 1.1508%). Source: FRED DTB3.

27.3. Return distribution

Hide

```
ggplot(data.frame(Return = r_p_para), aes(x = Return)) +
  geom_density(fill = "#2196F3", color = "#1565C0", alpha = 0.3, linewidth = 0.8) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey40") +
  geom_vline(xintercept = mean(r_p_para), linetype = "solid", color = "#1565C0", linewidth = 0.8) +
  labs(title    = "Daily Return Distribution — Tangency Portfolio",
       subtitle = "Blue vertical line = mean return",
       x = "Log-Return", y = "Density") +
  theme_minimal(base_size = 12) +
  theme(plot.title    = element_text(face = "bold", hjust = 0.5),
        plot.subtitle = element_text(hjust = 0.5, color = "grey40"))
```



28. Backtesting report

Basel III requires VaR backtesting over a 250 trading-day window at the 99% confidence level (alpha = 1%) on a 1-day horizon. The number of exceptions (actual loss > VaR) determines the regulatory capital multiplier via the Traffic Light framework:

Hide

```
# ----------------------------------------------------------------
# Traffic Light classification per model & distribution
# Basel reference: alpha = 1%, 250-day OOS window
# ----------------------------------------------------------------
traffic_light <- function(n_exceptions, T_oos = 250) {
  p_vals <- sapply(0:n_exceptions, function(k)
    dbinom(k, T_oos, prob = 0.01))
  cum_p  <- sum(p_vals)   # P(X <= n_exceptions | p=0.01)

  zone   <- dplyr::case_when(
    n_exceptions <= 4  ~ "Green",
    n_exceptions <= 9  ~ "Yellow",
    TRUE               ~ "Red"
  )
  add_on <- dplyr::case_when(
    n_exceptions == 0  ~ 0.00,
    n_exceptions == 1  ~ 0.00,
    n_exceptions == 2  ~ 0.00,
    n_exceptions == 3  ~ 0.00,
    n_exceptions == 4  ~ 0.00,
    n_exceptions == 5  ~ 0.40,
    n_exceptions == 6  ~ 0.50,
    n_exceptions == 7  ~ 0.65,
    n_exceptions == 8  ~ 0.75,
    n_exceptions == 9  ~ 0.85,
    TRUE               ~ 1.00
  )
  data.frame(
    Exceptions     = n_exceptions,
    Zone           = zone,
    CumProb        = round(cum_p, 4),
    Multiplier_k   = round(3 + add_on, 2),
    stringsAsFactors = FALSE
  )
}

# Apply to all models at alpha = 1%
bt_1pct <- all_results[all_results$Alpha == 0.01, ]

basel_tbl <- do.call(rbind, lapply(1:nrow(bt_1pct), function(i) {
  row  <- bt_1pct[i, ]
  tl   <- traffic_light(row$N_Violation)
  data.frame(
    Model        = row$Model,
    Dist         = row$Dist,
    Exceptions   = tl$Exceptions,
    Exc_Rate     = row$Exc_Rate,
    Zone         = tl$Zone,
    CumProb      = tl$CumProb,
    k_multiplier = tl$Multiplier_k,
    UC_pval      = row$UC_pval,
    CC_pval      = row$CC_pval,
    DQ_pval      = row$DQ_pval,
    stringsAsFactors = FALSE
  )
}))

# Colour-code zones
zone_colors <- ifelse(basel_tbl$Zone == "Green",  "#C8E6C9",
               ifelse(basel_tbl$Zone == "Yellow", "#FFF9C4", "#FFCDD2"))

kable(basel_tbl,
      caption = "Basel III Traffic Light Backtesting Report — alpha = 1%, T = 250 days") %>%
  kable_styling(bootstrap_options = c("striped","hover","condensed","bordered"),
                full_width = FALSE, font_size = 12) %>%
  column_spec(5, background = zone_colors) %>%
  footnote(general = paste0(
    "Green zone: 0-4 exceptions (k=3.00). ",
    "Yellow zone: 5-9 exceptions (k=3.40 to 3.85). ",
    "Red zone: 10+ exceptions (k=4.00). ",
    "CumProb = P(X <= exceptions | p=0.01, T=250). ",
    "Higher k = higher regulatory capital charge."),
    general_title = "Basel III Note: ")
```

Basel III Traffic Light Backtesting Report — alpha = 1%, T = 250 days

Model	Dist	Exceptions	Exc_Rate	Zone	CumProb	k_multiplier	UC_pval	CC_pval	DQ_pval
DCC-GARCH	Normal	4	0.016	Green	0.8922	3.0	0.3805	0.0873	0.0074
DCC-GARCH	Student-t	3	0.012	Green	0.7581	3.0	0.7580	0.9194	0.9912
GO-GARCH	Normal	6	0.024	Yellow	0.9863	3.5	0.0594	0.0503	0.0132
GO-GARCH	Student-t	3	0.012	Green	0.7581	3.0	0.7580	0.9194	0.9936
Parametric	Normal	6	0.024	Yellow	0.9863	3.5	0.0594	0.0029	0.0000
Parametric	Student-t	3	0.012	Green	0.7581	3.0	0.7580	0.9194	0.9681
Historical Simulation	Empirical	4	0.016	Green	0.8922	3.0	0.3805	0.0873	0.0036

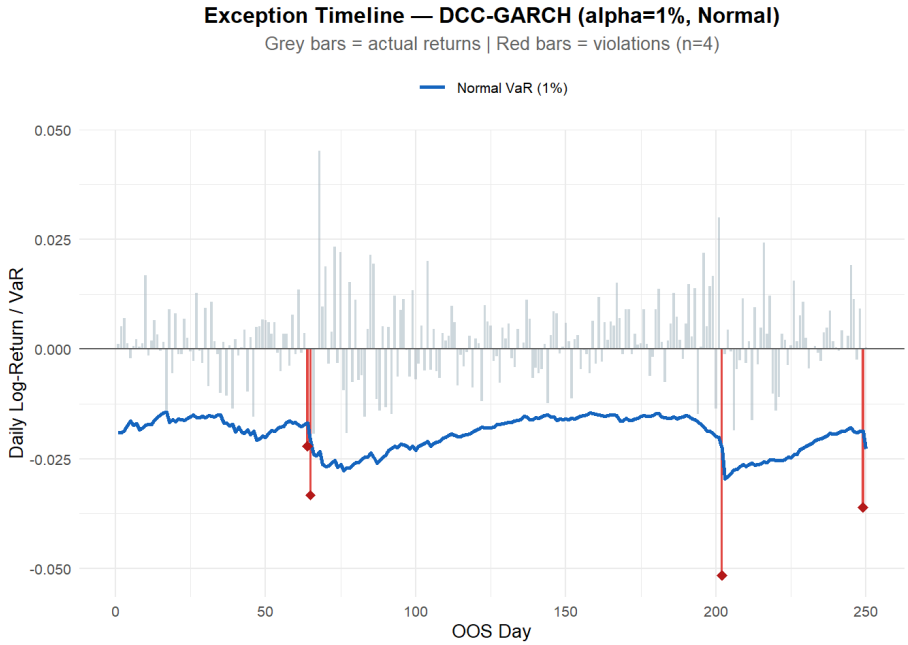
Basel III Note:

Green zone: 0-4 exceptions (k=3.00). Yellow zone: 5-9 exceptions (k=3.40 to 3.85). Red zone: 10+ exceptions (k=4.00). CumProb = P(X <= exceptions | p=0.01, T=250). Higher k = higher regulatory capital charge.

Hide

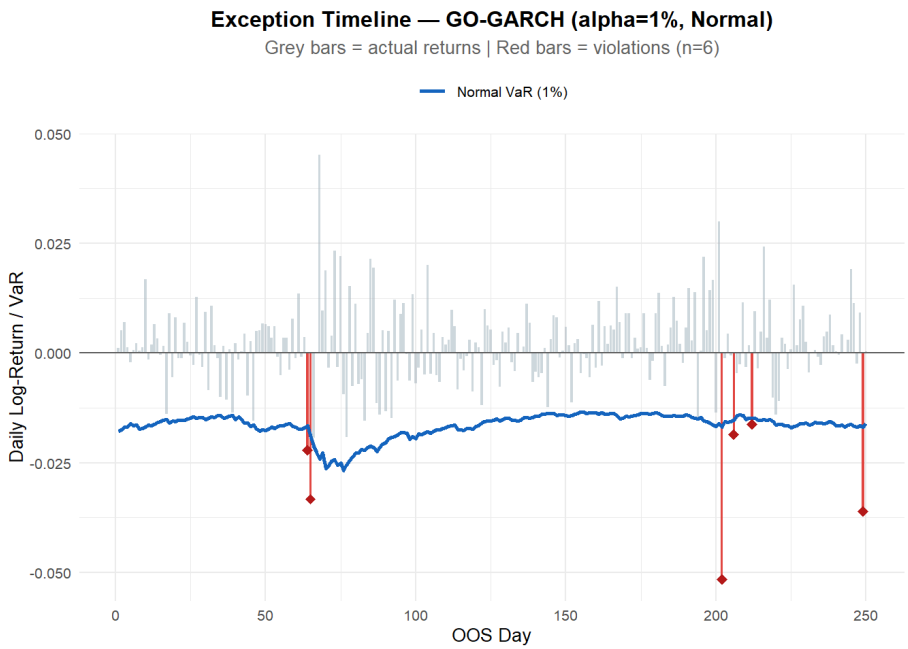
```
# ---------------------------------------------------------------
# Daily exceptions timeline — shows WHEN violations occur
# Clustering of violations is a warning sign (CC test failure)
# ---------------------------------------------------------------
res_dcc_1  <- compute_var_es_param(mu_para_roll, sigma_dcc,        0.01, df_t)
res_go_1   <- compute_var_es_param(mu_para_roll, sigma_go,         0.01, df_t)
res_para_1 <- compute_var_es_param(mu_para_roll, sigma_para_roll,  0.01, df_t)

# ---------------------------------------------------------------
# Student-t VaR at df=5 for each parametric model
# qt(0.01, df=5) replaces qnorm(0.01) in VaR computation
# ---------------------------------------------------------------
df_stress <- 5
t_scalar  <- qt(0.01, df = df_stress) / qnorm(0.01)  # scaling factor vs Normal

VaR_t5_dcc  <- mu_para_roll + qt(0.01, df = df_stress) * sigma_dcc
VaR_t5_go   <- mu_para_roll + qt(0.01, df = df_stress) * sigma_go
VaR_t5_para <- mu_para_roll + qt(0.01, df = df_stress) * sigma_para_roll

# ---------------------------------------------------------------
# Reusable exception timeline function
# mode: "Normal" or "Student-t (df=5)" — controls label & color
# ---------------------------------------------------------------
plot_exc_timeline <- function(actual, VaR, label, mode = "Normal") {

  line_color <- if (mode == "Normal") "#1565C0" else "#6A1B9A"
  line_label <- if (mode == "Normal") "Normal VaR (1%)" else "Student-t VaR (1%, df=5)"

  df <- data.frame(
    t         = seq_along(actual),
    Return    = actual,
    VaR       = VaR,
    violation = actual < VaR
  )

  ggplot(df, aes(x = t)) +
    geom_bar(aes(y = Return), stat = "identity",
             fill = "#B0BEC5", alpha = 0.6, width = 0.8) +
    geom_bar(data = df[df$violation, ], aes(y = Return),
             stat = "identity", fill = "#E53935", alpha = 0.9, width = 0.8) +
    geom_line(aes(y = VaR, color = line_label), linewidth = 0.9) +
    geom_point(data = df[df$violation, ],
               aes(x = t, y = Return),
               color = "#B71C1C", size = 2.5, shape = 18) +
    geom_hline(yintercept = 0, color = "grey40", linewidth = 0.4) +
    scale_color_manual(values = setNames(line_color, line_label)) +
    labs(
      title    = sprintf("Exception Timeline — %s (alpha=1%%, %s)", label, mode),
      subtitle = sprintf(
        "Grey bars = actual returns | Red bars = violations (n=%d)",
        sum(df$violation)
      ),
      x = "OOS Day", y = "Daily Log-Return / VaR", color = NULL
    ) +
    theme_minimal(base_size = 10) +
    theme(
      plot.title    = element_text(face = "bold", hjust = 0.5),
      plot.subtitle = element_text(hjust = 0.5, color = "grey40"),
      legend.position = "top"
    )
}

# ---------------------------------------------------------------
# HS VaR at alpha=1%
# ---------------------------------------------------------------
VaR_hs_1pct <- numeric(n_out)
for (t in 1:n_out) {
  day_OOS        <- start_out + t - 1
  idx_end        <- day_OOS - 1
  idx_start      <- idx_end - 250 + 1
  w_t            <- W_tan[t, ]
  port_w         <- as.numeric(returns[idx_start:idx_end, ] %*% w_t)
  VaR_hs_1pct[t] <- quantile(port_w, probs = 0.01, type = 7)
}

# ---------------------------------------------------------------
```

```
# Plot — Normal VaR (4 models)
# ------------------------------------------------------------
print(plot_exc_timeline(r_p_para, res_dcc_1$VaR_N,  "DCC-GARCH",     "Normal"))
```



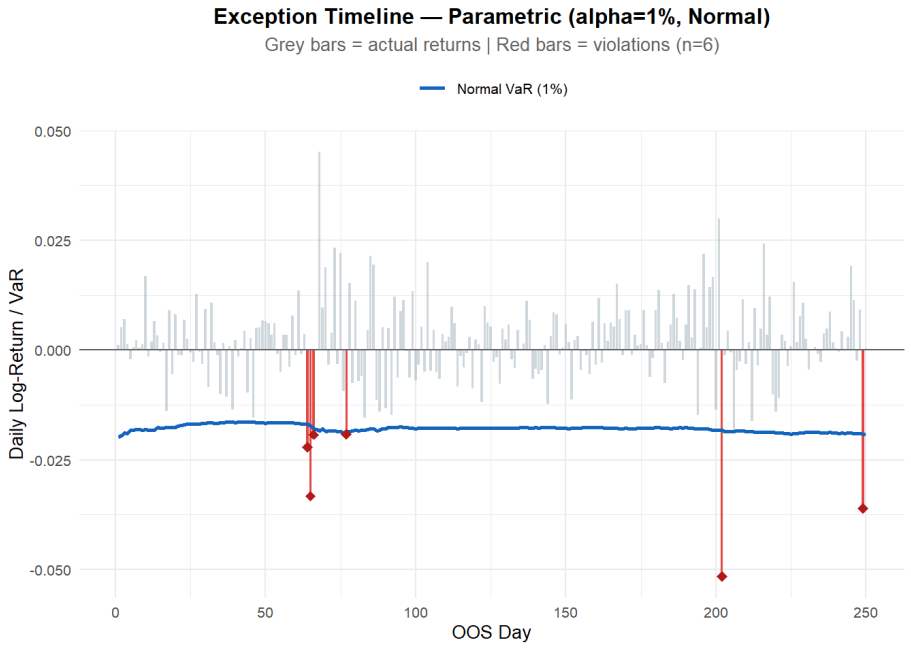
Hide

```
print(plot_exc_timeline(r_p_para, res_go_1$VaR_N,   "GO-GARCH",      "Normal"))
```



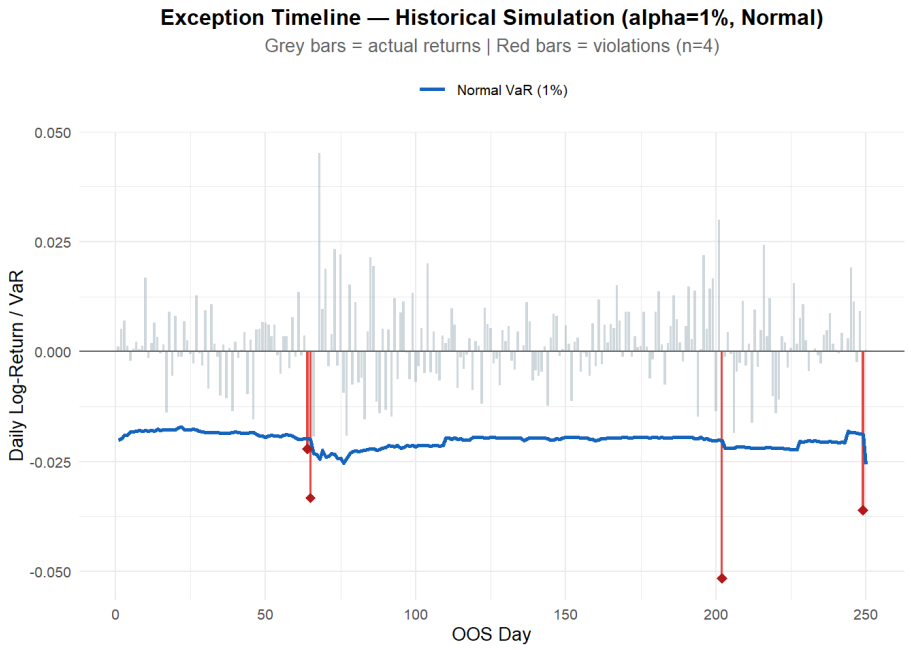
Hide

```
print(plot_exc_timeline(r_p_para, res_para_1$VaR_N, "Parametric",    "Normal"))
```



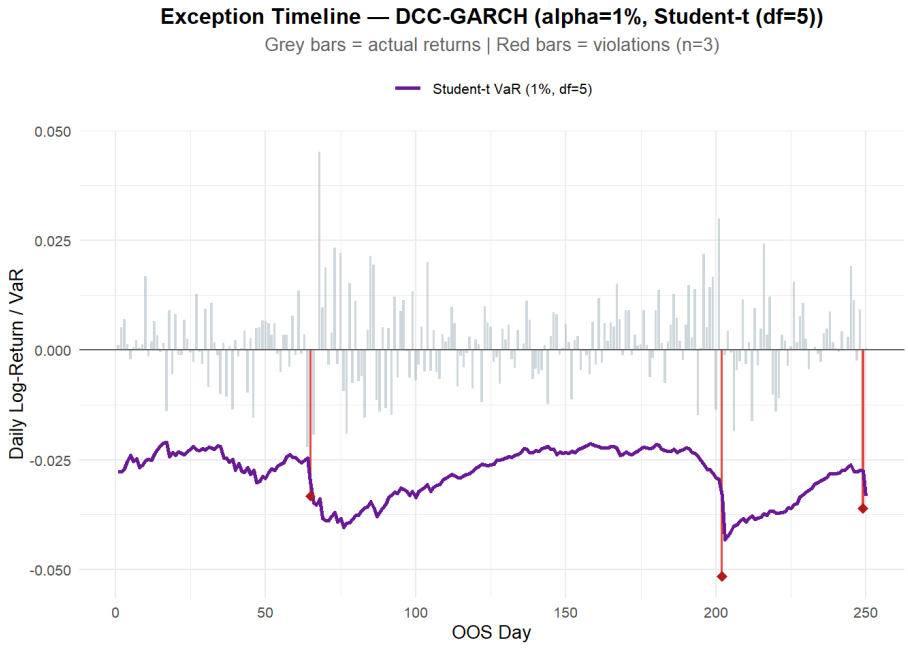
Hide

```
print(plot_exc_timeline(r_p_para, VaR_hs_1pct,      "Historical Simulation"))
```



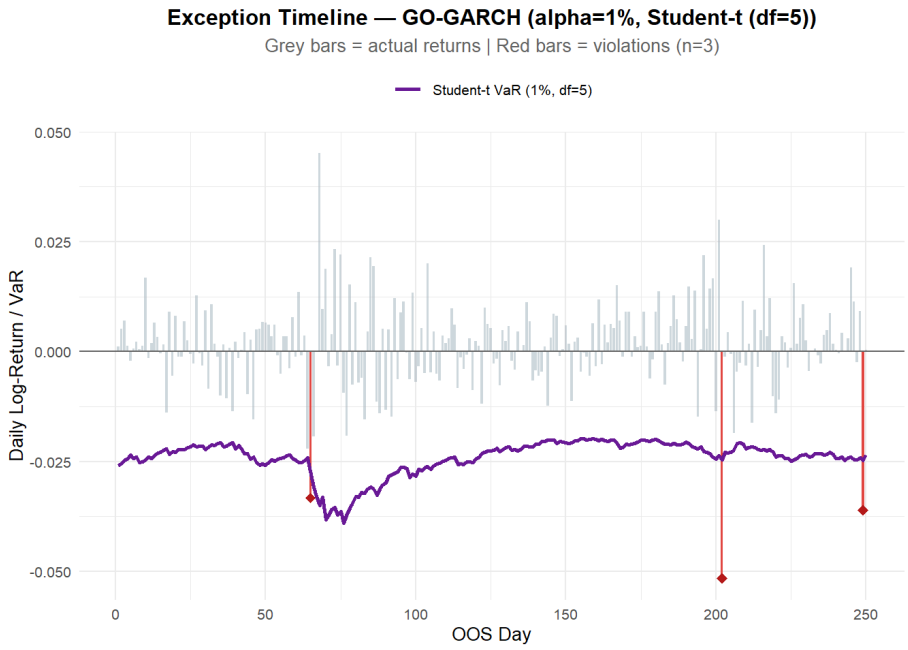
Hide

```
# ----------------------------------------------------------------
# Plot — Student-t VaR df=5 (3 parametric models, HS không áp dụng)
# Student-t có đuôi dày hơn Normal → VaR thấp hơn (conservative hơn)
# qt(0.01, df=5) ≈ -3.365 vs qnorm(0.01) ≈ -2.326 → ~45% deeper
# ----------------------------------------------------------------
print(plot_exc_timeline(r_p_para, VaR_t5_dcc,  "DCC-GARCH",  "Student-t (df=5)"))
```



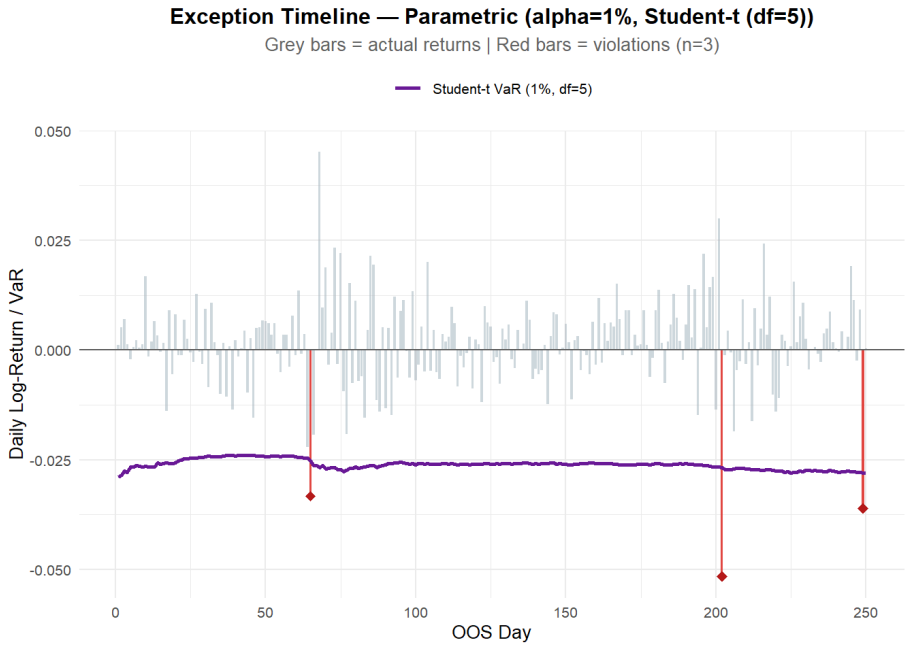
Hide

```
print(plot_exc_timeline(r_p_para, VaR_t5_go,   "GO-GARCH",   "Student-t (df=5)"))
```



Hide

```
print(plot_exc_timeline(r_p_para, VaR_t5_para, "Parametric", "Student-t (df=5)"))
```


Exception Timeline — Parametric (alpha=1%, Student-t (df=5))
Grey bars = actual returns | Red bars = violations (n=3)
Student-t VaR (1%, df=5)
Daily Log-Return / VaR
0.050
0.025
0.000
-0.025
-0.050
0
50
100
150
200
250
OOS Day